Orchestrated Inference

Structured Reasoning Through Tool Composition in VDR-LLM-Prolog

Registry: [HOWL-VDR-9-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → [HOWL-VDR-3-2026] → [HOWL-VDR-4-2026] → [HOWL-LLM-1-2026] → [HOWL-VDR-5-2026] → [HOWL-VDR-6-2026] → [HOWL-VDR-7-2026] → [HOWL-VDR-8-2026] → [HOWL-VDR-9-2026]

DOI: 10.5281/zenodo.20217696

Date: May 2026

Domain: Applied Philosophy / Systems Architecture / Structured Inference

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

The prior papers in this series built a language model architecture with exact arithmetic (VDR-1 through VDR-4), scoped knowledge bases with constraints and provenance (VDR-5), 333 deterministic primitives invoked through compact command tokens (VDR-6, VDR-8), a complete model lifecycle as KB operations (VDR-7), runtime data primitives for working memory (VDR-8), universal dotted-path addressing (VDR-8), and session snapshots with disposable cloning (VDR-8). Each paper added a capability. None of them specified how those capabilities compose into multi-step inference processes.

This paper specifies Orchestrated Inference — the pattern by which the language model uses its tools to conduct structured investigations that produce traceable, quantified conclusions. The language model does not reason. It orchestrates. It selects and sequences exact tools — Prolog for logical deduction, Python for numerical computation, pure primitives for data manipulation, operational primitives for external data acquisition — in a loop that produces inferences neither the language model nor any single tool could produce alone.

The paper defines the orchestrated inference loop (assess → formalize → execute → store → assess), inference notebooks as standard KB schemas for housing investigations, four inference modes (deductive, inductive, abductive, analogical) with their characteristic tool signatures, external data integration patterns for bringing real-world data into the exact system, and inference provenance that gives every conclusion a complete, queryable derivation chain with exact confidence scores.

The central claim is precise: the language model predicts tokens; the tools compute and deduce; the composition produces structured inferences; the KB records everything. Orchestrated Inference is not artificial reasoning. It is a reasoning exoskeleton — external structure that compensates for the language model's computational unreliability while leveraging its strength at pattern recognition, intent mapping, and natural language formalization.

---

1. What Orchestrated Inference Is and Is Not

1.1 What It Is

Orchestrated Inference is a pattern of tool composition. The language model recognizes what kind of problem it faces, translates the problem's structure into formal representations, dispatches exact tools to compute and deduce, stores results in the KB, assesses the new state, and repeats. Each step is small. Each step uses the right tool for that step's job. The composition of steps produces a structured inference that is traceable from conclusion back to raw evidence.

The language model's contribution is orchestration — deciding what to do next, what formalism to use, what data to gather, when to backtrack, when to conclude. This is pattern recognition over structured state. It is what language models do well.

The tools' contribution is computation and deduction — sorting lists, computing exact statistics, evaluating Prolog queries, executing Python scripts, fetching external data. These are deterministic operations. They produce the same output from the same input every time. They are what tools do well.

Neither the language model nor the tools can conduct a multi-step investigation alone. The language model cannot reliably sort a list, compute a correlation coefficient, or evaluate a logical rule chain. The tools cannot recognize that a latency spike might be caused by a database connection leak, or that a research question requires gathering evidence from multiple sources before ranking hypotheses. The composition covers what each component lacks.

1.2 What It Is Not

Orchestrated Inference does not make the language model smarter. The language model still predicts tokens. It still hallucinates. It still makes errors in judgment. What the pattern does is reduce the impact of those errors and amplify the value of correct judgments.

When the language model correctly identifies that a problem needs transitive closure over a dependency graph, the Prolog engine computes it exactly. When the language model incorrectly identifies the wrong approach, the exact tools produce an exact wrong answer from wrong premises — but the provenance chain shows exactly what happened, the constraints may catch the error, and the backtracking mechanism provides recovery.

Orchestrated Inference does not guarantee correct conclusions. Deductive conclusions

from correct premises are guaranteed by Prolog. But the premises might be wrong — the language model chose bad rules. The evidence might be incomplete — the language model did not query the right source. The inductive scoring might be flawed — the language model's weighting criteria might miss the actual cause. The paper specifies how these failure modes are detectable through the provenance chain, not how they are prevented. Prevention is the language model's responsibility. Detection is the system's.

---

2. The Orchestrated Inference Loop

2.1 The Five Phases

The loop has five phases that cycle until termination:

Assess. The language model reads the current state — the problem statement, the evidence gathered so far, the KB contents, the data primitive states (counters, queues, stacks, LRU caches), the pending goals — and determines what kind of step is needed next. This is the pattern-matching phase. The language model examines structured state and decides: do I need more evidence? Should I formalize a hypothesis? Should I query Prolog? Should I write Python? Should I backtrack? Should I conclude?

Formalize. The language model translates the needed step into an executable form. If logical deduction is needed, it writes Prolog rules and asserts facts. If numerical analysis is needed, it writes a Python script. If data transformation is needed, it assembles a chain of pure primitive invocations. If external data is needed, it constructs the appropriate operational command. The formalization is the creative act — a small, targeted program for one step of the inference.

Execute. The system runs the formalized step. Prolog evaluates the query. Python runs in the sandbox. The primitive computes. The external API returns data. Execution is deterministic (for pure operations) or logged and grant-gated (for operational ones). The language model does not execute. The tools do.

Store. The result goes into a KB location — a working data binding, an LRU entry, a Prolog fact, a counter update, a ring buffer write. The result is now addressable by dotted path, persistent within the session, and available for subsequent steps. Storage includes provenance: what tool produced this result, from what inputs, at what turn.

Assess again. The loop returns to the assessment phase. The language model reads the new state, including the result just stored, and determines the next step.

2.2 Loop Termination

The loop terminates under four conditions:

Goal satisfaction. The language model queries Prolog for the investigation's declared goal, and the goal is satisfied. For an SRE incident, the goal might be `root_cause(X, Confidence)` where Confidence exceeds a threshold. For a

research compilation, the goal might be `ranked_approaches(List, CoverageScore)` where coverage exceeds a threshold. Goal satisfaction is checked at every assessment phase.

Budget exhaustion. Every inference notebook has resource budgets tracked by counters: maximum steps, maximum external queries, maximum Python executions, maximum wall-clock time. When a budget counter reaches its limit, the loop transitions to conclude with whatever partial findings exist, or to halt if no meaningful conclusion is possible.

Stall detection. A counter tracks how many loop iterations have passed since the last new piece of evidence was stored. If this counter exceeds a configurable threshold (default 5), the loop has stalled — it is consuming resources without making progress. Stall triggers either a backtrack (try a different approach) or a forced conclusion.

User intervention. The user can cancel an investigation, redirect it, or request a premature conclusion at any time. User commands override loop state.

2.3 Loop Resource Management

The loop manages its own resource consumption through the data primitives from VDR-8:

```
counter("steps_executed")      // total loop iterations

counter("queries_issued")      // external API/DB queries

counter("scripts_executed")      // Python script runs

counter("hypotheses_tested")    // distinct hypotheses evaluated

counter("steps_since_evidence")  // stall detector
```

At every assessment phase, the language model checks these counters against the notebook's declared budget constraints. The constraints are standard VDR-5 constraints on the notebook KB:

```
constraint("step_budget", operational, active,

condition(counter_get("steps_executed") < 50),

on_violation("conclude_partial"))

constraint("query_budget", operational, active,

condition(counter_get("queries_issued") < 20),

on_violation("no_more_external"))
```

```
constraint("stall_detection", operational, active,

condition(counter_get("steps_since_evidence") < 5),

on_violation("backtrack_or_conclude"))
```

When the language model stores new evidence, it resets the stall counter:

```
CMD: counter_reset(root.inference.notebook_001.steps_since_evidence)
```

When the language model completes a step without new evidence, it increments the stall counter:

```
CMD: counter_inc(root.inference.notebook_001.steps_since_evidence)
```

The budget system is not imposed externally. It is part of the notebook KB, governed by the same constraint system that governs everything else in the VDR-LLM-Prolog architecture. The language model can query its own budget, plan accordingly, and make trade-offs — spend the remaining query budget on one comprehensive Prometheus query or several targeted ones.

2.4 Backtracking

When a step fails, or evidence contradicts the current hypothesis, or the stall detector fires, the loop backtracks. The investigation path is a stack (VDR-8 bounded stack) that records the current exploration direction:

```
CMD: stack_push(root.inference.notebook_001.investigation_path,

"testing hypothesis: database connection leak")
```

Backtracking pops the stack, records the abandoned path in the LRU of attempted approaches (preventing duplicate work), and assesses from the previous branch point:

```
CMD: stack_pop(root.inference.notebook_001.investigation_path)

→ "testing hypothesis: database connection leak"

CMD: lru_push(root.inference.notebook_001.attempted_steps,

"db_connection_leak", "abandoned: contradicted by connection count data")
```

Before formalizing a new approach, the language model checks the LRU:

```
CMD: lru_contains(root.inference.notebook_001.attempted_steps,
```

```
"network_saturation") → false

// Not tried yet — proceed
```

This prevents the loop from revisiting failed approaches. The LRU holds a bounded history of what was tried and why it was abandoned. The stack holds the current path with branch points for backtracking. Together they give the loop memory of its own exploration.

2.5 Branching

When the assessment phase identifies a separable sub-problem, the loop can spawn a child notebook:

```
CMD: KB_ASSERT(root.inference.notebook_001.children,

child_notebook("root.inference.notebook_001.sub_correlation"))
```

The child notebook inherits the parent's persistent facts (through KB scoping) but gets its own data primitives and its own budget allocation drawn from the parent's remaining budget. The child investigates its sub-problem independently and returns its conclusion to the parent. The parent's loop resumes at the assessment phase with the child's conclusion available as a new fact.

---

3. Inference Notebooks

3.1 What a Notebook Is

An inference notebook is not a new data structure. It is a KB subtree with a declared schema — specific facts, rules, data primitives, and constraints that collectively house one inference process. The notebook uses the existing KB struct from VDR-8 without modification.

The notebook is the unit of inference. One investigation, one notebook. The notebook holds the problem statement, the plan, the evidence gathered, the intermediate results, the rules written, the conclusions reached, and the full trace of how each conclusion was derived. When the investigation is complete, the notebook is archived as a permanent record.

3.2 Notebook Structure

```
KB: root.inference.incident_002

  path: root.inference.incident_002
```

```
  // Problem definition
  facts:
problem_statement("API latency P99 exceeded 5000ms threshold")
inference_type(abductive)
goal(root_cause(_, Confidence), Confidence > fraction(80, 100))
status(active)
created_at(timestamp(2026, 5, 16, 14, 35, 0))

  // Plan and navigation
  queue: step_queue (capacity: 30)
  stack: investigation_path (capacity: 15)

  // Evidence tracking
  lru: findings (capacity: 30)
  lru: sources (capacity: 20)
  lru: attempted_steps (capacity: 50)

  // Budget
  counter: steps_executed (max: 50)
  counter: queries_issued (max: 20)
  counter: scripts_executed (max: 10)
  counter: hypotheses_tested
```

```
  counter: evidence_count
  counter: steps_since_evidence (max: 5)

  // Coordination
  lock: investigation_active

  // Completion tracking
  bitset: evidence_dimensions (width: task-specific)

  // Rolling context
  ring: metric_snapshots (capacity: 60)

  // Reasoning (populated during investigation)
 rules: (Prolog rules written by the LLM during investigation)

  // Conclusions (populated at end)
  facts: (conclusion facts with derivation references)

  // Constraints
  constraints:
constraint("step_budget", ...)
constraint("query_budget", ...)
constraint("stall_detection", ...)
```

```
constraint("minimum_evidence", operational, active,
    condition(counter_get("evidence_count") >= 3),
    on_violation("warn_before_conclude"))

  // Connections
  connections:
connection(target: root.ops.monitoring_prod_v1,
   relationship: "evidence_source", direction: inbound)
connection(target: root.ops.sre_rules,
   relationship: "domain_knowledge", direction: inbound)

  // Children (sub-investigations if any)
 children: [root.inference.incident_002.correlation_analysis, ...]
```

3.3 Notebook Lifecycle

A notebook passes through four states:

Active. The investigation is in progress. The loop is cycling. Data primitives are being mutated. Evidence is accumulating. The lock is held.

Concluded. The investigation has reached one or more conclusions. The conclusions are asserted as persistent facts with full provenance. The data primitives preserve their final state. The lock is released. The notebook remains queryable.

Halted. The investigation stopped without a conclusion — budget exhaustion, unrecoverable error, user cancellation. Partial findings are preserved. The halt reason is recorded. The notebook remains queryable.

Archived. The notebook is frozen. All state is preserved read-only. The notebook is moved under root.archive.inference but remains fully queryable. This is the permanent record.

3.4 Notebook Templates

Common investigation types have pre-defined templates — standard schemas with appropriate data primitives, constraints, initial step queues, and pre-loaded Prolog rules. The language model instantiates a template and customizes it for the specific problem.

Templates are themselves KBs stored under root.templates.inference. They are not special — they are ordinary KBs whose facts describe the schema for a type of investigation. Creating a notebook from a template is a KB copy operation that populates the new notebook's structure.

Available templates and their specifications are detailed in the appendix tables.

---

4. The Four Inference Modes

4.1 Deductive Inference

Given premises and rules, derive what must be true.

The language model asserts known facts and writes Prolog rules that encode logical relationships. Prolog evaluates queries against the facts and rules, producing conclusions that follow necessarily from the premises. The conclusions are guaranteed correct if the premises and rules are correct — the Prolog engine's unification and backtracking are exact.

Characteristic tool signature:

```
KB_ASSERT (facts) → KB_ASSERT (rules) → KB_QUERY → exact conclusion
```

Example: The language model knows that a deployment changed a configuration parameter, and knows a rule that reducing a capacity parameter below current load causes saturation. It asserts both and queries Prolog for the consequence:

```
CMD: KB_ASSERT(root.inference.notebook,

fact(config_change(max_connections, 8192, 4096, at(14:15))))

CMD: KB_ASSERT(root.inference.notebook,

fact(current_load(connections, 4200)))

CMD: KB_ASSERT(root.inference.notebook,

rule(causes_saturation(Param, OldVal, NewVal) :-

    config_change(Param, OldVal, NewVal, _),
```

```
    current_load(Param, Current),
    Current > NewVal))
CMD: KB_QUERY(root.inference.notebook,
causes_saturation(max_connections, 8192, 4096))
→ true
```

The conclusion is exact. If the premises are true, saturation is a necessary consequence.

Confidence propagation: Deductive conclusions inherit the minimum confidence of their premises. If all premises are exact VDR facts (confidence 1/1) and all rules are logically valid (confidence 1/1), the conclusion has confidence 1/1. If one premise came from an external source with confidence 95/100, the conclusion's confidence is at most 95/100. The weakest link determines the chain's strength.

4.2 Inductive Inference

Given observations, propose what hypothesis best explains or characterizes the data.

The language model gathers evidence from multiple sources, asserts observations as facts, writes Prolog rules that score hypotheses by how much evidence supports each one, and ranks candidates. The conclusions are probable, not certain — they depend on the evidence being representative and the scoring criteria being appropriate.

Characteristic tool signature:

```
External data gathering → KB_ASSERT (evidence) → Prolog scoring rules →
list_sort_by_key → ranked hypotheses with coverage scores
```

Example: The language model is compiling research on memory reduction techniques. It has gathered 8 approaches from various sources, each with a memory reduction percentage, a quality impact, and a source reliability score. It writes scoring rules:

```
CMD: KB_ASSERT(root.inference.notebook,
rule(score(Approach, Score) :-
   approach(Approach, memory_reduction(MR), quality_impact(QI), source_type
    source_weight(ST, SW),
    Score is MR * SW - QI * fraction(2, 1)))
```

```
CMD: KB_QUERY(root.inference.notebook,

findall(A-S, score(A, S), Scores))

CMD: PURE_FN list_sort_by_key(Scores, key(second), descending)

→ ranked approaches
```

Confidence propagation: Inductive confidence is the product of evidence coverage and mean source confidence. If 7 out of 10 relevant dimensions have been checked (coverage 7/10), and the mean source confidence is 90/100, the inductive confidence is 63/100. Gathering more evidence increases coverage. Using more reliable sources increases mean confidence.

4.3 Abductive Inference

Given an observation, infer the most likely cause.

This is diagnosis — working backward from symptoms to explanations. The language model writes Prolog rules encoding causal relationships (if X then Y), asserts the observed symptoms, and queries for which causes could produce the observed effects. It then gathers discriminating evidence to narrow the candidates, re-queries, and ranks by explanation completeness.

Characteristic tool signature:

```
KB_ASSERT (observations + causal rules) → KB_QUERY (findall explanations) →

External probes for discriminating evidence → KB_ASSERT (new evidence) →

KB_QUERY (re-evaluate) → Python correlation → ranked causes
```

Example: The language model observes a latency spike. It writes causal rules:

```
CMD: KB_ASSERT(root.inference.notebook,

rule(possible_cause(upstream_timeout) :-

    symptom(error_type(502)),

    symptom(latency_p99_elevated)))

CMD: KB_ASSERT(root.inference.notebook,

rule(possible_cause(deployment_regression) :-
```

```
symptom(error_rate_spike(Service, started_at(T))),
 recent_deployment(Service, deployed_at(D)),
 abs(T - D) < minutes(30)))
```

It queries for possible causes, gets two candidates, then writes Python to pull Prometheus data and compute cross-correlation between the latency series and database connection count series. The correlation result (an exact VDR fraction) discriminates between hypotheses. It asserts the correlation as evidence and re-queries. One cause now has more evidence. The causal chain is recorded.

Confidence propagation: Abductive confidence is based on explanation completeness — what fraction of the observed symptoms does the leading hypothesis explain? If the hypothesis explains 4 out of 5 symptoms with evidence from reliable sources, the confidence reflects that coverage. Abductive inference also tracks alternative explanations and their scores, because a "best explanation" with a close second is less confident than one that dominates.

4.4 Analogical Inference

Given a known domain and an unfamiliar domain, identify structural parallels and transfer conclusions.

The language model writes Prolog facts describing the structure of a well-understood domain and the partial structure of an unfamiliar domain. It writes rules that match structural patterns — if domain A has the relationship R(X, Y) and domain B has entities X' and Y' that map to X and Y, then R(X', Y') is hypothesized in domain B. The strength of the analogy is the fraction of source-domain relationships that have matching target-domain counterparts.

Characteristic tool signature:

```
KB_ASSERT (source domain structure) → KB_ASSERT (target domain structure) →
Prolog structural matching → analogy strength score →
transferred conclusions with degraded confidence
```

Example: The language model knows how circuit breaker patterns work in electrical engineering (overload triggers break, break isolates fault, system continues with reduced capacity). A software system is exhibiting similar behavior — high load causes a component to stop accepting requests, which isolates the failing service. The language model writes both structures as Prolog facts, writes matching rules, and queries for the structural correspondence. The analogy suggests that the software system has an implicit circuit breaker and will auto-recover when load drops — a transferred conclusion from the electrical domain.

Confidence propagation: Analogical confidence is the product of the analogy strength (matched relationships / total relationships) and the source domain confidence. A strong analogy (90% structural match) from a well-established domain (95/100 confidence) produces transferred conclusions at 85.5/100. A weak analogy (50% match) produces conclusions at 47.5/100 — speculative, not actionable without verification.

4.5 Mode Composition

Real investigations use multiple modes. A typical incident investigation starts with abductive inference (what could cause these symptoms?), gathers evidence using operational primitives, uses inductive inference to rank hypotheses by evidence fit, then uses deductive inference to derive the implications of the leading hypothesis (if the cause is X, then action Y should fix it). The modes compose naturally because they all operate on the same KB, use the same tools, and produce facts in the same format.

The language model selects modes based on the assessment phase's reading of the current state. If there are symptoms without explanation, it defaults to abductive. If there are observations without patterns, it defaults to inductive. If there are premises and rules ready to evaluate, it defaults to deductive. If there is a well-understood domain and an unfamiliar parallel, it defaults to analogical. Mode selection is a pattern-matching task — exactly what the language model is good at.

---

5. External Data Integration

5.1 The Integration Pipeline

Orchestrated inference rarely operates on data already in the KB. It needs external data — monitoring metrics, API responses, database query results, file contents, search results. The standard pipeline for integrating external data has six stages:

Acquire. An operational primitive retrieves raw data from an external source. This is grant-gated and logged per VDR-6.

Parse. Pure primitives extract structure from the raw response. parse_json for JSON APIs, parse_csv for tabular data, string_split for log files.

Convert. Pure primitives convert external values to exact VDR types. This is the conversion boundary — the point where approximate external data enters the exact internal system. The conversion is declared and logged.

Store. KB_ASSERT places the structured, converted data at a dotted path in the inference notebook.

Index. Data primitives make the data accessible for fast lookup. lru_push for recent findings, ring_write for rolling time series windows, bitset_set for tracking which sources have been checked.

Process. Pure primitives or Python scripts perform analytical operations — statistical summaries, sorting, filtering, correlation — producing derived values that are stored back into the KB.

5.2 Prometheus Integration Example

This is the most detailed example because it illustrates every stage of the pipeline and shows how external time series data flows through the system into Prolog reasoning.

```
// Acquire

CMD: OP_FN net_fetch(

"http://prometheus:9090/api/v1/query_range?query=http_request_duration_sec

grant: "monitoring_read")

→ stored at root.inference.notebook.raw_latency

CMD: counter_inc(root.inference.notebook.queries_issued)

// Parse

CMD: PURE_FN parse_json(root.inference.notebook.raw_latency) → parsed

CMD: PURE_FN dict_get(parsed, "data") → data_block

CMD: PURE_FN dict_get(data_block, "result") → result_list

CMD: PURE_FN list_nth(result_list, 0) → first_series

CMD: PURE_FN dict_get(first_series, "values") → timestamp_value_pairs

// Convert

CMD: PURE_FN list_map(timestamp_value_pairs,

fn(pair, [to_number(pair[0]), to_fraction(pair[1])])) → exact_series

CMD: KB_ASSERT(root.inference.notebook.conversion_boundary,

fact(conversion("prometheus_float", "vdr_fraction",

   method("decimal_string_to_fraction"), max_error(fraction(0, 1)))))
```

```
// Store
CMD: KB_ASSERT(root.inference.notebook.timeseries,
binding("latency_p99", exact_series))
// Index
CMD: PURE_FN for_each(exact_series, fn(point,
ring_write(root.inference.notebook.metric_snapshots, point)))
CMD: bitset_set(root.inference.notebook.evidence_dimensions, 0)
CMD: counter_inc(root.inference.notebook.evidence_count)
CMD: counter_reset(root.inference.notebook.steps_since_evidence)
CMD: lru_push(root.inference.notebook.sources, "prometheus_latency",
"P99 latency, 60 data points, 1-minute resolution")
// Process
CMD: PURE_FN list_map(exact_series, fn(pair, pair[1])) → values_only
CMD: PURE_FN stat_mean(values_only) → fraction(4200, 1)
CMD: PURE_FN stat_percentile(values_only, fraction(99, 100)) → fraction(8200
CMD: PURE_FN list_filter(exact_series,
predicate(fn(pair, pair[1] > fraction(4000, 1)))) → spike_points
CMD: PURE_FN list_head(spike_points) → [spike_onset_timestamp, first_spike_v
CMD: KB_ASSERT(root.inference.notebook.stats,
binding("mean_latency_ms", fraction(4200, 1)))
CMD: KB_ASSERT(root.inference.notebook.stats,
binding("spike_onset", spike_onset_timestamp))
CMD: lru_push(root.inference.notebook.findings, "latency_profile",
```

```
"mean 4200ms, spike onset at 14:27, 38 minutes above threshold")
```

Every value that entered the system from Prometheus is now an exact VDR fraction. Every subsequent operation on this data — statistical analysis, Prolog reasoning, Python correlation — operates on exact values. The conversion boundary is recorded. The provenance chain from raw Prometheus response to computed statistic is complete.

5.3 The Conversion Boundary

When external float data enters the exact system, a conversion boundary fact is asserted. This is the system's declaration that an approximation has entered the exact layer:

```
CMD: KB_ASSERT(root.inference.notebook.conversion_boundary,

fact(conversion(

    source("prometheus_float"),

    target("vdr_fraction"),

    original("4237.5"),

    converted(fraction(8475, 2)),

    method("decimal_string_to_fraction"),

    max_error(fraction(0, 1)),

    turn(47))))
```

For terminating decimals, the conversion is exact (max_error = 0/1). For repeating decimals or values with declared precision limits, the max_error is bounded and recorded. The conversion boundary ensures that the provenance chain never silently introduces approximation — every point where external imprecision enters is declared.

5.4 Multi-Source Correlation

When an investigation pulls data from multiple external sources, the language model needs to correlate them. The standard approach:

1. Acquire all relevant time series through separate Prometheus queries (or API calls).
2. Parse and convert each to exact VDR fractions.
3. Store each at a distinct dotted path in the notebook.
4. Write Python to compute cross-correlation, because correlation requires operations (products of deviations from means, sums of squares) that are easier to express as a script than as a chain of primitive invocations.

5. Store the correlation results (exact fractions) back in the KB.
6. Assert the correlations as evidence facts for Prolog reasoning.

```
// After multiple Prometheus queries and processing:
CMD: ENV_UPLOAD(env_analysis, correlate.py, root.inference.scripts.correlat
CMD: ENV_EXEC(env_analysis, "python3", root.inference.scripts.correlate)
→ {latency_vs_db: fraction(94, 100), latency_vs_cpu: fraction(78, 100),
  latency_vs_rps: fraction(12, 100), db_leads_latency_by: 3}
CMD: KB_ASSERT(root.inference.notebook.evidence,
fact(correlation(db_connections, latency_p99, fraction(94, 100), lag(3))))
CMD: KB_ASSERT(root.inference.notebook.evidence,
fact(correlation(cpu_usage, latency_p99, fraction(78, 100), lag(0))))
CMD: KB_ASSERT(root.inference.notebook.evidence,
fact(correlation(request_rate, latency_p99, fraction(12, 100), lag(0))))
```

Now Prolog reasons over the correlations:

```
CMD: KB_ASSERT(root.inference.notebook.reasoning,
rule(leading_indicator(Metric) :-
   correlation(Metric, latency_p99, Strength, lag(Lag)),
    Strength > fraction(9, 10),
    Lag > 0))
CMD: KB_QUERY(root.inference.notebook.reasoning, leading_indicator(X))
→ X = db_connections
```

The external data has been acquired, parsed, converted to exact fractions, correlated by Python, stored in the KB, and reasoned over by Prolog. Each step used the right tool. The complete chain from raw Prometheus JSON to logical conclusion is in the notebook.

---

6. Inference Provenance

6.1 What Provenance Means for Inference

Every conclusion produced by orchestrated inference carries a complete derivation chain. This is not new — VDR-5 specified provenance for all KB facts. What VDR-9 specifies is the provenance schema specific to multi-step inference processes, where the derivation chain passes through multiple tools, multiple inference modes, and multiple sources.

6.2 Conclusion Record

When the loop concludes, the final inference is asserted as a structured fact:

```
CMD: KB_ASSERT(root.inference.notebook.conclusion,

fact(inference_conclusion(

    id("conclusion_001"),

    statement(root_cause(db_connection_exhaustion,

        triggered_by(deployment_v2_3_1))),

    mode(abductive),

    confidence(fraction(92, 100)),

    derived_from([

      evidence("prometheus_latency", confidence(fraction(95, 100))),

      evidence("prometheus_db_connections", confidence(fraction(95, 100))),

      evidence("correlation_analysis", confidence(fraction(90, 100))),

      evidence("deployment_api", confidence(fraction(98, 100))),

      evidence("cache_inspection", confidence(fraction(95, 100)))

    ]),

    via_rules([

        "possible_cause/1", "leading_indicator/1",

        "full_chain/5", "root_cause/2"
```

```
    ]),
    via_tools([
      "net_fetch(prometheus)  $\times$ 6", "parse_json  $\times$ 6",
      "stat_mean  $\times$ 4", "list_filter  $\times$ 3",
      "ENV_EXEC(correlate.py)  $\times$ 1", "ENV_EXEC(check_cache.py)  $\time
    ]),
    alternatives([
      alternative(network_saturation, confidence(fraction(15, 100)),
          reason("request_rate correlation only 12%")),
      alternative(deployment_code_bug, confidence(fraction(25, 100)),
         reason("timing matches but no code path to latency"))
    ]),
    steps_taken(23),
    external_queries(6),
    backtracks(1),
    notebook("root.inference.incident_002")
)))
```

6.3 Confidence Computation

Confidence is an exact VDR fraction computed from the provenance chain according to declared rules. The confidence of a conclusion depends on the inference mode that produced it and the confidences of its inputs.

Deductive conclusions: confidence = min(premise confidences). The weakest premise determines the chain's strength.

Inductive conclusions: confidence = evidence_coverage $\times$ mean(source confidences). Partial evidence gives partial confidence.

Abductive conclusions: confidence = (explained_symptoms / total_symptoms) × min(evidence confidences for best explanation). Unexplained symptoms degrade confidence.

Analogical conclusions: confidence = analogy_strength × source_domain_confidence. Weak analogies degrade transferred knowledge.

Multi-mode chains: When modes compose (abductive generates hypothesis, deductive derives implication, inductive scores against evidence), the overall confidence is the minimum across mode transitions. Each mode junction is a potential weakness.

All confidence computations use exact VDR fractions. The confidence of a conclusion is not a vague label ("high," "medium," "low") — it is a fraction like 92/100, computed from declared rules, traceable to specific evidence confidences, and exactly reproducible by replaying the computation.

6.4 Challenging a Conclusion

A user can challenge any conclusion by asserting a counter-fact:

```
CMD: KB_ASSERT(root.inference.notebook.challenge,

fact(counter_evidence("The deployment at 14:15 was rolled back at 14:20,

    before the symptoms started at 14:27")))
```

The system re-evaluates the affected parts of the derivation chain. If the counter-fact invalidates a premise, the deductive conclusions that depend on that premise are flagged. If the counter-fact provides evidence against the leading abductive hypothesis, the hypothesis ranking is recomputed. The re-evaluation is a Prolog query over the existing rules and facts plus the new counter-fact:

```
CMD: KB_ASSERT(root.inference.notebook.evidence,

fact(deployment_rolled_back(v2_3_1, at(14:20))))

CMD: KB_QUERY(root.inference.notebook.reasoning,

root_cause(db_connection_exhaustion, triggered_by(deployment_v2_3_1)))
```

If the rules account for rollback (a rolled-back deployment can still leave artifacts like cached config), the conclusion may survive. If the rules do not, the query fails and the conclusion is retracted or its confidence is degraded. Either way, the challenge and its impact are recorded in the notebook.

---

7. Worked Example: SRE Incident Investigation

This section traces a complete orchestrated inference through all phases, showing every command token, every mode transition, and every data primitive interaction.

7.1 Setup

A monitoring watch fires: API gateway P99 latency exceeds 5000ms. The language model creates an inference notebook from the SRE template:

```
CMD: lock_acquire(root.inference.incident_002.investigation_active,

holder: "latency_spike_investigation")

CMD: counter_create(root.inference.incident_002.steps_executed)

CMD: counter_create(root.inference.incident_002.queries_issued)

CMD: counter_create(root.inference.incident_002.hypotheses_tested)

CMD: counter_create(root.inference.incident_002.evidence_count)

CMD: counter_create(root.inference.incident_002.steps_since_evidence)

CMD: lru_create(root.inference.incident_002.findings, capacity: 30)

CMD: lru_create(root.inference.incident_002.attempted_steps, capacity: 50)

CMD: lru_create(root.inference.incident_002.sources, capacity: 20)

CMD: queue_create(root.inference.incident_002.step_queue, capacity: 30)

CMD: stack_create(root.inference.incident_002.investigation_path, capacity:

CMD: bitset_create(root.inference.incident_002.evidence_dimensions, width:

CMD: ring_create(root.inference.incident_002.metric_snapshots, capacity: 60

CMD: KB_ASSERT(root.inference.incident_002,

fact(problem_statement("API gateway P99 latency exceeded 5000ms")))

CMD: KB_ASSERT(root.inference.incident_002,

fact(inference_type(abductive)))
```

```
CMD: KB_ASSERT(root.inference.incident_002,
fact(goal(root_cause(_, Confidence), Confidence > fraction(80, 100))))
```

The language model loads the initial plan into the step queue:

```
CMD: queue_push(root.inference.incident_002.step_queue,
"pull latency time series from Prometheus")
CMD: queue_push(root.inference.incident_002.step_queue,
"pull correlated metrics (CPU, DB, request rate)")
CMD: queue_push(root.inference.incident_002.step_queue,
"check recent deployments")
CMD: queue_push(root.inference.incident_002.step_queue,
"formalize causal hypotheses")
CMD: queue_push(root.inference.incident_002.step_queue,
"compute cross-correlations")
CMD: queue_push(root.inference.incident_002.step_queue,
"test leading hypothesis with targeted probe")
CMD: queue_push(root.inference.incident_002.step_queue,
"confirm root cause")
```

7.2 Loop Iteration 1: Acquire Latency Data

Assess: Language model pops the step queue. First step: pull latency data.

```
CMD: queue_pop(root.inference.incident_002.step_queue)
→ "pull latency time series from Prometheus"
CMD: counter_inc(root.inference.incident_002.steps_executed)
```

Formalize: Construct the Prometheus query URL.

Execute: Fetch, parse, convert, store, index, process — the full pipeline from Section 5.2.

Store: Latency time series at root.inference.incident_002.timeseries.latency_p99. Statistics at root.inference.incident_002.stats. Finding in LRU.

```
CMD: counter_inc(root.inference.incident_002.evidence_count)

CMD: counter_reset(root.inference.incident_002.steps_since_evidence)

CMD: bitset_set(root.inference.incident_002.evidence_dimensions, 0)

CMD: lru_push(root.inference.incident_002.findings, "latency_profile",

"mean 4200ms, P99 8200ms, spike onset at 14:27")
```

7.3 Loop Iteration 2: Acquire Correlated Metrics

Assess: Next step in queue. Pull CPU, DB connections, and request rate from Prometheus.

```
CMD: queue_pop(root.inference.incident_002.step_queue)

→ "pull correlated metrics"

CMD: counter_inc(root.inference.incident_002.steps_executed)
```

Formalize, Execute, Store: Three Prometheus queries, each through the full pipeline. Three time series stored. Three evidence dimensions checked.

```
CMD: counter_add(root.inference.incident_002.queries_issued, 3)

CMD: counter_add(root.inference.incident_002.evidence_count, 3)

CMD: counter_reset(root.inference.incident_002.steps_since_evidence)

CMD: bitset_set(root.inference.incident_002.evidence_dimensions, 1)  // CPU

CMD: bitset_set(root.inference.incident_002.evidence_dimensions, 2)  // DB

CMD: bitset_set(root.inference.incident_002.evidence_dimensions, 3)  // RPS
```

7.4 Loop Iteration 3: Check Deployments

Assess: Next step. Check recent deployments.

```
CMD: queue_pop(root.inference.incident_002.step_queue)
→ "check recent deployments"
CMD: counter_inc(root.inference.incident_002.steps_executed)
```

Formalize: Write a Python script to query the deployment API.

Execute:

```
CMD: ENV_UPLOAD(env_ops, check_deploy.py, root.inference.scripts.deploy)
CMD: ENV_EXEC(env_ops, "python3", root.inference.scripts.deploy)
→ deployment(api_gateway, v2.3.1, deployed_at(14:15))
```

Store:

```
CMD: KB_ASSERT(root.inference.incident_002.evidence,
fact(recent_deployment(api_gateway, v2_3_1, deployed_at(14:15))))
CMD: counter_inc(root.inference.incident_002.evidence_count)
CMD: counter_reset(root.inference.incident_002.steps_since_evidence)
CMD: bitset_set(root.inference.incident_002.evidence_dimensions, 4)
CMD: lru_push(root.inference.incident_002.findings, "deployment",
"v2.3.1 deployed at 14:15, 12 minutes before spike onset")
```

7.5 Loop Iteration 4: Formalize Hypotheses (Mode: Abductive)

Assess: Evidence gathered. Time to formalize causal hypotheses.

```
CMD: queue_pop(root.inference.incident_002.step_queue)
→ "formalize causal hypotheses"
CMD: counter_inc(root.inference.incident_002.steps_executed)
CMD: stack_push(root.inference.incident_002.investigation_path,
"formalizing causal hypotheses from gathered evidence")
```

Formalize: Write Prolog causal rules from SRE domain knowledge:

```
CMD: KB_ASSERT(root.inference.incident_002.reasoning,
rule(possible_cause(upstream_timeout) :-
    symptom(latency_p99_elevated),
    symptom(error_type(502))))
CMD: KB_ASSERT(root.inference.incident_002.reasoning,
rule(possible_cause(deployment_regression) :-
   symptom(error_rate_spike(Service, started_at(T))),
    recent_deployment(Service, _, deployed_at(D)),
    abs(T - D) < 30))
CMD: KB_ASSERT(root.inference.incident_002.reasoning,
rule(possible_cause(dependency_failure) :-
    symptom(error_type(502)),
    depends_on(api_gateway, Dep),
    health_check_failing(Dep)))
CMD: KB_ASSERT(root.inference.incident_002.reasoning,
rule(possible_cause(resource_exhaustion) :-
    symptom(latency_p99_elevated),
    resource_near_limit(_, Utilization),
    Utilization > fraction(90, 100)))
```

Execute: Query for matching hypotheses:

```
CMD: KB_QUERY(root.inference.incident_002.reasoning,
findall(C, possible_cause(C), Causes))
```

```
→ Causes = [upstream_timeout, deployment_regression]
```

Only two match — dependency_failure needs a health_check_failing fact (not yet gathered), resource_exhaustion needs a resource_near_limit fact (not yet gathered). Prolog tells the language model what it can conclude now and implicitly what's missing.

Store:

```
CMD: KB_ASSERT(root.inference.incident_002.hypotheses,

fact(hypothesis_set_v1([upstream_timeout, deployment_regression],

   missing_for([dependency_failure, resource_exhaustion]))))

CMD: counter_set(root.inference.incident_002.hypotheses_tested, 2)

CMD: lru_push(root.inference.incident_002.findings, "initial_hypotheses",

"2 matching, 2 need more data")
```

7.6 Loop Iteration 5: Cross-Correlation (Mode: Inductive)

Assess: Two hypotheses match, two need data. Before gathering more data, compute correlations to rank what we have. This is a mode switch from abductive to inductive.

```
CMD: queue_pop(root.inference.incident_002.step_queue)

→ "compute cross-correlations"

CMD: counter_inc(root.inference.incident_002.steps_executed)
```

Formalize: Write Python for cross-correlation analysis.

Execute:

```
CMD: ENV_UPLOAD(env_analysis, correlate.py, root.inference.scripts.correlat

CMD: ENV_EXEC(env_analysis, "python3", root.inference.scripts.correlate)

→ {db_connections_correlation: fraction(94, 100),

   db_leads_by_minutes: 3,

   cpu_correlation: fraction(78, 100),
```

```
   request_rate_correlation: fraction(12, 100)}
```

Store:

```
CMD: KB_ASSERT(root.inference.incident_002.evidence,

fact(correlation(db_connections, latency_p99, fraction(94, 100), lag_minutes

CMD: KB_ASSERT(root.inference.incident_002.evidence,

fact(correlation(request_rate, latency_p99, fraction(12, 100), lag_minutes(0

CMD: counter_inc(root.inference.incident_002.evidence_count)

CMD: counter_reset(root.inference.incident_002.steps_since_evidence)

CMD: lru_push(root.inference.incident_002.findings, "correlation",

"DB connections correlate 94% with 3-min lead. Request rate: 12% —
not traffic.")
```

7.7 Loop Iteration 6: Targeted Probe (Mode: Abductive, Refined)

Assess: DB connections correlate strongly and lead the latency spike by 3 minutes. This is consistent with resource_exhaustion, which needs a resource_near_limit fact. The language model probes specifically for this.

```
CMD: queue_pop(root.inference.incident_002.step_queue)

→ "test leading hypothesis with targeted probe"

CMD: counter_inc(root.inference.incident_002.steps_executed)

CMD: stack_push(root.inference.incident_002.investigation_path,

"testing: DB connection exhaustion as root cause")
```

Formalize and Execute: Pull database connection pool state from Prometheus:

```
CMD: OP_FN net_fetch(

"http://prometheus:9090/api/v1/query?query=pg_stat_activity_count&time=...

grant: "monitoring_read")
```

```
CMD: counter_inc(root.inference.incident_002.queries_issued)
// Parse, convert, store...
CMD: KB_ASSERT(root.inference.incident_002.evidence,
fact(resource_near_limit(db_connections, fraction(99, 100))))
CMD: KB_ASSERT(root.inference.incident_002.evidence,
fact(idle_in_transaction_count(153)))
CMD: KB_ASSERT(root.inference.incident_002.evidence,
fact(max_connections(200)))
```

Re-query Prolog with new evidence:

```
CMD: KB_QUERY(root.inference.incident_002.reasoning,
findall(C, possible_cause(C), Causes))
→ Causes = [upstream_timeout, deployment_regression, resource_exhaustion]
```

resource_exhaustion now matches. The language model writes a refinement rule:

```
CMD: KB_ASSERT(root.inference.incident_002.reasoning,
rule(explains_all(resource_exhaustion) :-
    possible_cause(resource_exhaustion),
   correlation(db_connections, latency_p99, Corr, lag_minutes(Lag)),
    Corr > fraction(9, 10),
    Lag > 0))
CMD: KB_QUERY(root.inference.incident_002.reasoning,
explains_all(resource_exhaustion))
→ true
```

7.8 Loop Iteration 7: Confirm Root Cause (Mode: Deductive)

Assess: resource_exhaustion explains all symptoms, correlates 94% with the leading metric, and the leading metric (DB connections) is at 99% capacity. The language model ties this to the deployment with a deductive chain.

```
CMD: queue_pop(root.inference.incident_002.step_queue)

→ "confirm root cause"

CMD: counter_inc(root.inference.incident_002.steps_executed)
```

Formalize: Write the full causal chain as Prolog:

```
CMD: KB_ASSERT(root.inference.incident_002.reasoning,

rule(root_cause(db_connection_exhaustion_from_deployment, Confidence) :-

   recent_deployment(api_gateway, Version, deployed_at(DeployTime)),

    resource_near_limit(db_connections, Utilization),

    Utilization > fraction(95, 100),

   correlation(db_connections, latency_p99, Corr, lag_minutes(Lag)),

    Corr > fraction(9, 10),

    Lag > 0,

    Confidence is Corr * fraction(95, 100)))
```

Execute:

```
CMD: KB_QUERY(root.inference.incident_002.reasoning,

root_cause(Cause, Confidence))

→ Cause = db_connection_exhaustion_from_deployment,

  Confidence = fraction(8930, 10000)
```

Confidence: 89.3/100. Above the 80/100 threshold declared in the goal.

Check goal satisfaction:

```
CMD: KB_QUERY(root.inference.incident_002,
goal(root_cause(_, C), C > fraction(80, 100)))
→ satisfied
```

7.9 Conclusion

The loop concludes. The language model asserts the conclusion with full provenance:

```
CMD: KB_ASSERT(root.inference.incident_002.conclusion,
fact(inference_conclusion(
    statement(root_cause(db_connection_exhaustion,
       triggered_by(deployment(v2_3_1, at(14:15))))),
    mode(abductive),
    confidence(fraction(893, 1000)),
    derived_from([
     evidence("prometheus_latency", fraction(95, 100)),
     evidence("prometheus_db_connections", fraction(95, 100)),
       evidence("prometheus_cpu", fraction(95, 100)),
       evidence("prometheus_rps", fraction(95, 100)),
       evidence("deployment_api", fraction(98, 100)),
     evidence("correlation_analysis", fraction(90, 100))
    ]),
    alternatives([
     alternative(network_saturation, fraction(12, 100)),
      alternative(code_regression, fraction(25, 100))
```

```
    ]),
    steps_taken(7),
    external_queries(6),
    backtracks(0)
)))
CMD: lock_release(root.inference.incident_002.investigation_active)
CMD: KB_ASSERT(root.inference.incident_002, fact(status(concluded)))
```

The investigation produced a traceable conclusion in 7 loop iterations and 6 Prometheus queries. Every step is in the notebook. Every fact has provenance. The confidence is an exact fraction derived from declared rules. The alternatives are recorded with their scores. The entire investigation is replayable by walking the notebook KB.

---

8. Worked Example: Programming Bug Investigation

A failing test. The language model creates a notebook and investigates.

```
CMD: lock_acquire(root.inference.bug_047.investigation_active, holder: "tes
CMD: lru_create(root.inference.bug_047.findings, capacity: 20)
CMD: stack_create(root.inference.bug_047.undo_stack, capacity: 10)
CMD: counter_create(root.inference.bug_047.retry_count)
CMD: bitset_create(root.inference.bug_047.resolved, width: 50)
```

Assess: Check recent failures for patterns:

```
CMD: lru_peek(root.project.bugs.recent_failures, 10)
→ [test_047: graph_traversal, test_044: graph_cycle, test_041: graph_bfs]
```

Three graph-related failures. The language model writes Prolog to find the common dependency:

```
CMD: KB_ASSERT(root.inference.bug_047.reasoning,
```

```
fact(calls(graph_traversal, graph_bfs)))
CMD: KB_ASSERT(root.inference.bug_047.reasoning,
fact(calls(graph_cycle, graph_bfs)))
CMD: KB_ASSERT(root.inference.bug_047.reasoning,
rule(common_dep(F1, F2, Dep) :- calls(F1, Dep), calls(F2, Dep), F1 \= F2))
CMD: KB_QUERY(root.inference.bug_047.reasoning,
common_dep(graph_traversal, graph_cycle, Dep))
→ Dep = graph_bfs
```

Formalize: Write an instrumented version. Save the current state for undo:

```
CMD: stack_push(root.inference.bug_047.undo_stack,
snapshot_ref("root.project.graph_module.v3"))
CMD: lock_acquire(root.project.graph_module.editing, holder: "bug_fix_047")
```

Execute: Run instrumented version, parse trace with string primitives:

```
CMD: ENV_EXEC(env_dev, "python3", root.project.scripts.trace_bfs)
→ trace output
CMD: PURE_FN string_split(trace_output, "\n") → lines
CMD: PURE_FN list_filter(lines, predicate(contains("ERROR"))) → error_lines
CMD: PURE_FN string_split(error_lines[0], ":")
→ ["line 31", "visited set not reset between calls"]
```

Root cause found. Write fix, run tests:

```
CMD: ENV_EXEC(env_dev, "python3 -m pytest test_graph.py")
→ 3 passed, 0 failed
```

Conclude:

```
CMD: bitset_set(root.inference.bug_047.resolved, 41)

CMD: bitset_set(root.inference.bug_047.resolved, 44)

CMD: bitset_set(root.inference.bug_047.resolved, 47)

CMD: lock_release(root.project.graph_module.editing)

CMD: lru_push(root.project.bugs.recent_fixes, "graph_bfs_visited_reset",

"shared mutable state: visited set not cleared between calls")
```

Write a permanent code smell rule:

```
CMD: KB_ASSERT(root.project.code_rules,

rule(smell(shared_mutable_state, Function) :-

    uses(Function, Collection),

    mutates(Function, Collection),

    called_multiple_times(Function),

    not(resets(Function, Collection))))
```

The investigation is concluded. Three bugs resolved from one root cause. The investigation produced a permanent detection rule. The fix is versioned, the undo stack has the rollback point, and the next time any function has a shared mutable state pattern, the code smell rule will catch it.

---

9. Worked Example: Research Compilation

User asks "list approaches to reducing memory usage in large language models." The language model creates a research notebook:

```
CMD: lru_create(root.inference.research_mem.papers, capacity: 50)

CMD: counter_create(root.inference.research_mem.papers_found)

CMD: counter_create(root.inference.research_mem.papers_analyzed)

CMD: queue_create(root.inference.research_mem.to_analyze, capacity: 30)
```

```
CMD: bitset_create(root.inference.research_mem.themes_covered, width: 10)
CMD: ring_create(root.inference.research_mem.methodology_notes, capacity: 2
CMD: lock_acquire(root.inference.research_mem.analysis_active, holder: "sur
```

Gather phase (inductive): Search, store findings, queue each for analysis:

```
CMD: KB_ASSERT(root.inference.research_mem.findings,
fact(approach("quantization",
    memory_reduction(fraction(75, 100)),
    quality_impact(fraction(2, 100)),
    source("Dettmers et al 2023"),
    source_type(peer_reviewed))))
CMD: counter_inc(root.inference.research_mem.papers_found)
CMD: queue_push(root.inference.research_mem.to_analyze, "quantization")
CMD: lru_push(root.inference.research_mem.papers, "quantization", "Dettmers
```

After gathering 8 approaches, process the analysis queue. For each approach, write Prolog relationships:

```
CMD: queue_pop(root.inference.research_mem.to_analyze) → "quantization"
CMD: KB_ASSERT(root.inference.research_mem.relationships,
fact(complementary(quantization, distillation)))
CMD: KB_ASSERT(root.inference.research_mem.relationships,
fact(conflicts(quantization, pruning_structured)))
CMD: ring_write(root.inference.research_mem.methodology_notes,
"quantization: experimental, controlled, tested up to 70B params")
CMD: counter_inc(root.inference.research_mem.papers_analyzed)
CMD: bitset_set(root.inference.research_mem.themes_covered, 0)
```

Scoring phase (inductive): Write Prolog scoring rules, query for ranked list:

```
CMD: KB_ASSERT(root.inference.research_mem.ranking,
rule(score(Approach, Score) :-
   approach(Approach, memory_reduction(MR), quality_impact(QI), _, source_ty
    source_weight(ST, SW),
    Score is MR * SW - QI * fraction(2, 1)))
CMD: KB_QUERY(root.inference.research_mem.ranking,
findall(A-S, score(A, S), Scores))
CMD: PURE_FN list_sort_by_key(Scores, key(second), descending) → ranked
```

Gap analysis (deductive): Find unreplicated claims:

```
CMD: KB_ASSERT(root.inference.research_mem.gap_analysis,
rule(unreplicated(Claim, Paper) :-
    paper_claims(Paper, Claims),
    member(claim(Claim, _), Claims),
   findall(P2, (paper_claims(P2, C2), member(claim(Claim, _), C2), P2 \= Pape
    Others = []))
CMD: KB_QUERY(root.inference.research_mem.gap_analysis,
findall(C-P, unreplicated(C, P), Gaps))
```

Combination analysis: Use Python to compute combined reduction for complementary pairs:

```
CMD: ENV_EXEC(env_dev, "python3", root.inference.scripts.combine)
→ {quantization+distillation: fraction(87, 100), ...}
```

Conclude: The ranked list has exact scores, source provenance, relationship analysis, gap identification, and combination effects. Coverage bitset shows which themes are represented:

```
CMD: bitset_to_list(root.inference.research_mem.themes_covered)

→ [0,1,2,3,5,6,7]  // themes 4, 8, 9 uncovered
```

The conclusion includes the coverage gap — the survey is explicit about what it did not find.

---

10. Worked Example: Argument Construction

User says "write a speech arguing for renewable energy investment." The language model creates an argument notebook:

```
CMD: queue_create(root.inference.speech_ren.argument_order, capacity: 20)

CMD: lru_create(root.inference.speech_ren.sources, capacity: 30)

CMD: counter_create(root.inference.speech_ren.unsupported_claims)

CMD: counter_create(root.inference.speech_ren.objections_addressed)

CMD: bitset_create(root.inference.speech_ren.evidence_dimensions, width: 8)
```

Evidence gathering (inductive): Search for supporting data, store as Prolog facts with source strength:

```
CMD: KB_ASSERT(root.inference.speech_ren.evidence,

fact(claim(cost_decline, "Solar costs declined 89% since 2010",

    source("IRENA 2024"), strength(peer_reviewed))))

CMD: lru_push(root.inference.speech_ren.sources, "cost_decline", "IRENA 2024

CMD: bitset_set(root.inference.speech_ren.evidence_dimensions, 0)

CMD: KB_ASSERT(root.inference.speech_ren.evidence,

fact(claim(job_creation, "Renewable sector employs 13.7M globally",

    source("blog post"), strength(unverified))))

CMD: counter_inc(root.inference.speech_ren.unsupported_claims)
```

The counter caught the weak source. The language model searches specifically for a better source for job creation data, finds the ILO report, updates:

```
CMD: KB_RETRACT(root.inference.speech_ren.evidence,
claim(job_creation, _, source("blog post"), _))
CMD: KB_ASSERT(root.inference.speech_ren.evidence,
fact(claim(job_creation, "Renewable sector employs 13.7M globally",
   source("ILO Global Report 2024"), strength(government_report))))
CMD: counter_dec(root.inference.speech_ren.unsupported_claims)
```

Argument structure validation (deductive): Write Prolog rules to verify the argument:

```
CMD: KB_ASSERT(root.inference.speech_ren.logic,
rule(well_supported(Claim) :-
    claim(Claim, _, _, strength(S)),
   member(S, [peer_reviewed, government_report, meta_analysis])))
CMD: KB_ASSERT(root.inference.speech_ren.logic,
rule(has_rebuttal(Objection) :-
    known_objection(_, Objection),
    rebuttal(Objection, _, _)))
CMD: KB_QUERY(root.inference.speech_ren.logic,
findall(C, (claim(C,_,_,_), not(well_supported(C))), Weak))
→ Weak = []  // all claims now supported
CMD: KB_QUERY(root.inference.speech_ren.logic,
findall(O, (known_objection(_, O), not(has_rebuttal(O))), Unaddressed))
→ Unaddressed = ["intermittency"]
```

One objection unaddressed. The language model searches for intermittency rebuttals,

finds battery storage data, asserts the rebuttal:

```
CMD: KB_ASSERT(root.inference.speech_ren.evidence,

fact(rebuttal("intermittency",

    "Grid-scale battery costs fell 97% since 1991, 4-
hour storage now cost-competitive",

    source("BloombergNEF 2024"))))

CMD: counter_inc(root.inference.speech_ren.objections_addressed)
```

Structure ordering: Build the argument flow in the queue:

```
CMD: queue_push(root.inference.speech_ren.argument_order,

"hook: energy cost impact on household budgets")

CMD: queue_push(root.inference.speech_ren.argument_order,

"claim: cost_decline — strongest data point first")

CMD: queue_push(root.inference.speech_ren.argument_order,

"claim: job_creation — economic benefits")

CMD: queue_push(root.inference.speech_ren.argument_order,

"preempt: intermittency — battery storage rebuttal")

CMD: queue_push(root.inference.speech_ren.argument_order,

"close: economic and environmental convergence")
```

Conclude: The language model generates the speech by popping sections from the queue. Each section references verified evidence from the KB. The final speech has zero unsupported claims, all known objections addressed, and a complete argument map in the notebook showing every claim, its source, its strength, and the logical structure.

```
CMD: counter_get(root.inference.speech_ren.unsupported_claims) → 0

CMD: KB_QUERY(root.inference.speech_ren.logic,

findall(O, (known_objection(_, O), not(has_rebuttal(O))), Gaps))
```

```
→ Gaps = []
```

---

11. Worked Example: Email Thread Analysis

User pastes a 15-message email thread. The language model creates an analysis notebook:

```
CMD: bitset_create(root.inference.email_001.messages_processed, width: 15)

CMD: lru_create(root.inference.email_001.key_statements, capacity: 50)

CMD: counter_create(root.inference.email_001.proposals_found)

CMD: counter_create(root.inference.email_001.open_questions)
```

Parse phase: String primitives extract structure from each message:

```
CMD: PURE_FN string_split(thread_text, "---BOUNDARY--
-") → messages

CMD: PURE_FN list_length(messages) → 15
```

For each message, extract sender, date, and key statements. Assert as Prolog facts:

```
CMD: PURE_FN string_split(messages[0], "\n") → lines

CMD: PURE_FN list_filter(lines, predicate(starts_with("From:"))) → sender_li

CMD: KB_ASSERT(root.inference.email_001.structure,

fact(message(0, sender("Sarah"), contains_proposal("migrate to new API by Q3'

CMD: KB_ASSERT(root.inference.email_001.structure,

fact(message(0, asks_question("what is the testing timeline?"))))

CMD: bitset_set(root.inference.email_001.messages_processed, 0)

CMD: counter_inc(root.inference.email_001.proposals_found)

CMD: counter_inc(root.inference.email_001.open_questions)

CMD: lru_push(root.inference.email_001.key_statements, "msg_0_proposal",
```

```
"Sarah proposes API migration by Q3")
```

Process all 15 messages. Verify completeness:

```
CMD: bitset_all_set(root.inference.email_001.messages_processed) → true
```

Analysis phase (deductive): Write Prolog rules for thread dynamics:

```
CMD: KB_ASSERT(root.inference.email_001.analysis,
rule(resolved_proposal(Proposal) :-
    message(_, _, contains_proposal(Proposal)),
   findall(A, (message(M, sender(A), affirms(Proposal)), M > 0), Agreements),
    length(Agreements, N), N >= 2,
    not((message(_, _, objects(Proposal, _))))))
CMD: KB_ASSERT(root.inference.email_001.analysis,
rule(contested_proposal(Proposal, Objections) :-
    message(_, _, contains_proposal(Proposal)),
   findall(A-R, (message(_, sender(A), objects(Proposal, R))), Objections),
    Objections \= []))
CMD: KB_ASSERT(root.inference.email_001.analysis,
rule(unanswered(Question, Asker) :-
   message(M, sender(Asker), asks_question(Question)),
    not((message(M2, _, answers(Question)), M2 > M))))
```

Query:

```
CMD: KB_QUERY(root.inference.email_001.analysis,
findall(P, resolved_proposal(P), Resolved))
→ Resolved = ["use staging for testing"]
```

```
CMD: KB_QUERY(root.inference.email_001.analysis,

findall(P-Os, contested_proposal(P, Os), Contested))

→ Contested = ["migrate to new API by Q3" -
[("Mike","too aggressive"),("Lisa","need compat")]]

CMD: KB_QUERY(root.inference.email_001.analysis,

findall(Q-A, unanswered(Q, A), Unanswered))

→ Unanswered = [("testing timeline?","Sarah"), ("who owns compat layer?","Lis
```

Conclude: One resolved decision, one contested proposal with two objections, two unanswered questions. Every conclusion traces to specific message numbers. The analysis is structural, not summarization — it was derived from Prolog rules applied to parsed facts, not from the language model's pattern-matched impression of the thread.

---

12. The Boundary Between Orchestration and Reasoning

12.1 What the Language Model Does

The language model performs five tasks in orchestrated inference:

Intent recognition. It reads the problem and determines what kind of investigation is needed — debugging, research, decision-making, diagnosis. This is pattern matching over natural language.

Mode selection. It assesses the current state and determines which inference mode (deductive, inductive, abductive, analogical) is appropriate for the next step. This is pattern matching over structured state.

Formalization. It translates problem structure into formal representations — Prolog rules, Python scripts, primitive chains. This is the creative act. The language model decides what rules to write, what to query, what script to construct. This is where the language model's training on code, logic, and domain knowledge contributes most.

Assessment. It reads results from the KB and determines what they mean — does this evidence support the hypothesis? Is the investigation stalled? Should we backtrack? This is judgment over structured data.

Framing. It generates natural language to present results to the user. This is text generation — the language model's native capability.

12.2 What the Language Model Does Not Do

The language model does not sort lists, compute statistics, evaluate logical rules, execute code, fetch external data, or perform any deterministic computation. Every one of these

operations is delegated to a tool that performs it correctly.

The language model does not hold the complete investigation state in its context window. Intermediate results are in the KB at dotted paths. The investigation path is on a stack. Recent findings are in an LRU. Progress is tracked by counters and bitsets. The language model reads these structures at each assessment phase — it does not need to remember them from earlier turns.

The language model does not guarantee that its formalizations are correct. It might write Prolog rules that do not capture the domain accurately. It might select the wrong inference mode. It might miss relevant evidence. These are the failure modes of orchestrated inference, and they are detectable — not preventable — through the provenance chain.

12.3 The Exoskeleton Metaphor

A physical exoskeleton does not replace the wearer's muscles. It amplifies them. The wearer decides to walk — the exoskeleton provides the force. The wearer decides to grip — the exoskeleton provides the strength.

Orchestrated Inference is a reasoning exoskeleton. The language model decides what to investigate — the tools provide the computation. The language model decides what rules to write — Prolog provides the deduction. The language model decides what data to gather — the operational primitives provide the access. The language model decides how to present results — the KB provides the exact data.

The exoskeleton does not think. The language model does not compute. Together they produce structured inferences that are traceable, reproducible, quantified, and inspectable. The composition is greater than either component alone. The boundary between them is clear and maintained.

---

13. Falsification Criteria

F1. If any deductive conclusion produced by Prolog does not follow necessarily from the asserted premises and rules, the Prolog engine has a bug. Testable by independent verification of the derivation.

F2. If any inference notebook's provenance chain has a gap — a conclusion that references evidence not present in the notebook KB — the provenance tracking is incomplete.

F3. If the confidence score propagated through a multi-step inference does not match the declared propagation rules applied to the input confidence scores, the confidence computation is wrong. Testable by exact VDR fraction comparison.

F4. If an external data value enters the inference system without a declared conversion boundary fact, the conversion tracking is incomplete.

F5. If a user challenges a conclusion by asserting a counter-fact, and the system does not re-evaluate the affected parts of the derivation chain, the challenge mechanism is broken.

F6. If the inference loop exceeds its declared resource budget without triggering the budget constraint, the budget enforcement is broken.

F7. If two inference notebooks operating on the same persistent KBs produce contradictory conclusions from the same evidence, and the contradiction is not detectable by querying both notebooks' conclusions, the contradiction detection is incomplete.

---

14. Conclusion

The eight prior papers built a system that can compute exactly, store knowledge in scoped KBs, execute deterministic primitives through command tokens, manage its own lifecycle, and maintain runtime working memory with session-level snapshots and cloning.

VDR-9 specifies how those capabilities compose into structured inference. The orchestrated inference loop cycles through assess, formalize, execute, store, and assess again — the language model selecting and sequencing exact tools that collectively produce traceable conclusions. Inference notebooks house investigations as standard KB schemas with data primitives for working memory, Prolog rules for logical structure, and provenance records for traceability. Four inference modes — deductive, inductive, abductive, analogical — have characteristic tool signatures that compose naturally for complex investigations. External data enters through a declared pipeline with conversion boundary tracking. Every conclusion carries an exact confidence score computed from declared propagation rules.

The language model does not reason. It orchestrates tools that compute and deduce. The orchestration is pattern recognition — deciding what to do next based on the current state. The computation and deduction are deterministic — exact tools producing exact results. The composition produces inferences that are inspectable at every step, challengeable at any conclusion, and reproducible by replaying the notebook.

Orchestrated Inference does not make the language model smarter. It makes its outputs more trustworthy by externalizing computation to exact tools, recording every step with provenance, quantifying confidence with exact fractions, and providing recovery through backtracking, budgets, and the entire session management infrastructure.

The reasoning exoskeleton is complete. The language model decides. The tools compute. The KB records. The provenance traces. The confidence quantifies. The constraints enforce. The notebooks preserve. Everything is queryable. Everything is exact. Everything is addressable by dotted path.

333 primitives. 4 inference modes. Notebooks as KB schemas. The orchestrated inference loop. Conversion boundaries. Confidence propagation. Backtracking with stack-based exploration. Budget management with counter-based constraints. Contradiction detection. Challenge mechanisms. Full provenance from raw evidence through logical derivation to quantified conclusion.

The system can now investigate.

END HOWL-VDR-9-2026

Registry: [HOWL-VDR-9-2026]

Status: Specification complete

Domain: Applied Philosophy / Systems Architecture / Structured Inference

Central Result: Orchestrated Inference — a specified pattern for multi-step, multi-tool, multi-mode inference with exact provenance, quantified confidence, and full traceability. The language model orchestrates; the tools compute and deduce; the KB records everything.

Foundation: VDR-1 through VDR-8, MATH-3, MATH-4

Key Claim: The language model does not reason. It orchestrates a reasoning process using exact tools. The composition produces structured inferences that are traceable, reproducible, quantified, and inspectable. Orchestrated Inference is a reasoning exoskeleton, not artificial reasoning.

Falsification: Seven specific criteria testable by provenance chain verification, exact confidence comparison, budget enforcement testing, and contradiction detection.

VDR-9 Extended Appendix Tables

Complete Reference Material for Orchestrated Inference

Appendix A: Inference Loop State Machine

A.1 Loop States

State	Description	Entry Condition	LLM Action	Tool Action
assess	Evaluate current evidence and determine next step	Loop start, or after store completes	Read KB state, data primitives, pending goals	None — LLM only
formalize	Translate needed step into executable form	Assessment identifies a gap or next action	Write Prolog rules, Python script, or primitive chain	None — LLM only
execute	Run the formalized step	Formalization complete	Issue command tokens	Tools execute deterministically

State	Description	Entry Condition	LLM Action	Tool Action
store	Persist results into KB and data primitives	Execution returns results	Issue KB ASSERT, lru push, counter inc, etc.	KB and primitives update
branch	Spawn sub-investigation	Assessment identifies separable sub-problem	Create child notebook, delegate	Child loop starts
backtrack	Abandon current path, try alternative	Execution fails, or evidence contradicts hypothesis	Pop investigation stack, load alternative	State restored from stack
conclude	Produce final inference with provenance	Goal satisfied, or budget exhausted, or no further steps available	Assert conclusion with derivation chain	Notebook marked concluded
halt	Terminate without conclusion	Unrecoverable error, or user cancels	Report status and partial findings	Notebook marked halted

A.2 State Transitions

From	To	Trigger	Logged As
assess	formalize	Gap identified, next step determined	assess complete(step type, rationale)
assess	conclude	All goals satisfied	goals satisfied(goal list)
assess	halt	Budget exhausted with no conclusion	budget exhausted(counter name, limit)
assess	branch	Sub-problem identified	branch created(child notebook path)
formalize	execute	Formalization complete (rules written, script ready)	formalized(artifact type, location)
formalize	backtrack	Cannot formalize current approach	formalization failed(reason)
execute	store	Execution returns result	executed(primitive or script, result summary)
execute	backtrack	Execution fails	execution failed(error type, detail)
store	assess	Result stored, ready for next assessment	stored(location, value summary)
branch	assess	Child notebook returns result	branch returned(child path, result)

From	To	Trigger	Logged As
backtrack	assess	Alternative loaded from stack	backtracked(abandoned path, new path)
conclude	(terminal)	—	conclusion(fact, confidence, derivation ref)
halt	(terminal)	—	halted(reason, partial findings ref)

A.3 Loop Invariants

Invariant	Condition	Verified When	On Violation
Step budget	counter get("steps executed") < max steps	Every transition to assess	Transition to halt
Query budget	counter get("queries issued") < max queries	Every transition to execute (external)	Skip external query, assess alternatives
Time budget	elapsed time < max duration	Every transition to assess	Transition to conclude (partial) or halt
Progress	steps since last new evidence < stall threshold	Every transition to assess	Warn, then backtrack if persists
Stack depth	stack size("investigation path") < max depth	Every transition to branch	Refuse branch, continue at current depth
No duplicate work	lru contains("attempted steps", current step) = false	Every transition to formalize	Skip step, assess alternative

---

Appendix B: Inference Notebook Schema

B.1 Required Fields

Field	Type	Created At	Purpose
problem statement	fact	Notebook creation	What is being investigated
inference type	atom	Notebook creation	Primary mode: deductive, inductive, abductive, analogical
goal	fact or list(fact)	Notebook creation	What constitutes a satisfactory conclusion
status	enum	Notebook creation, updated throughout	active, concluded, halted, branched
created at	timestamp	Notebook creation	When investigation started
concluded at	?timestamp	Conclusion	When investigation ended

B.2 Required Data Primitives

Primitive	Name Convention	Purpose	Typical Capacity
counter	steps executed	Loop budget tracking	max configurable
counter	queries issued	External query budget	max configurable
counter	hypotheses tested	Progress metric	no max
counter	evidence count	Evidence accumulation metric	no max
queue	step queue	Planned steps in execution order	30
stack	investigation path	Current exploration path with backtrack points	15
lru	findings	Recent discoveries for cross-reference	30

Primitive	Name Convention	Purpose	Typical Capacity
lru	attempted steps	Steps already tried to prevent duplicate work	50
lock	investigation active	Coordination signal	—
bitset	evidence gathered	Which sources or dimensions checked	task-dependent

B.3 Optional Fields

Field	Type	When Used	Purpose
parent notebook	path reference	Sub-investigations	Link to spawning notebook
child notebooks	[]path reference	Branched investigations	Links to spawned notebooks
templates used	[]atom	Template-based creation	Which templates contributed to this notebook's schema
deadline	timestamp	Time-bounded investigations	Hard stop time
stakeholders	[]atom	Collaborative inference	Who should be notified of conclusions
priority	enum (low, normal, high, critical)	Multi-notebook systems	Scheduling priority
tags	[]atom	Organization	User-defined classification
related incidents	[]path reference	SRE context	Links to related past investigations

B.4 Notebook Lifecycle States

State	step queue	investigation path	lock	Transitions To
created	May have initial steps	Empty	Not held	active

State	step queue	investigation path	lock	Transitions To
active	Being consumed	Growing	Held	concluded, halted, suspended
suspended	Preserved	Preserved	Released	active (on resume)
concluded	Empty or abandoned	Final path preserved	Released	archived
halted	May have remaining steps	Path at halt point preserved	Released	archived, active (on retry)
archived	Frozen	Frozen	Released	(terminal, but queryable)

Appendix C: Inference Mode Tool Signatures

C.1 Deductive Mode — Detailed Tool Chain

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
1. Assert premises	KB operations	KB ASSERT	Known facts	Facts in notebook KB	Source confidence inherited
2. Write rules	KB operations	KB ASSERT	Logical relationships	Rules in notebook KB	fraction(1,1) if logically valid
3. Query	KB operations	KB QUERY, findall	Goal predicate	Derived facts or failure	Minimum of premise confidences
4. Verify	Pure primitives	constraint check	Derived facts	Constraint status	fraction(1,1) if all pass
5. Chain	KB operations	depends on query	Conclusion	Full derivation chain	Weakest link in chain

C.2 Inductive Mode — Detailed Tool Chain

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
1. Gather evidence	External + pure	net fetch, parse json, string split	External sources	Structured data in KB	Source-dependent (see C.7)
2. Assert observations	KB operations	KB ASSERT	Parsed data	Evidence facts	Inherited from source
3. Write scoring rules	KB operations	KB ASSERT	Hypothesis-evidence relationships	Scoring rules	fraction(1,1) for rule structure
4. Score hypotheses	KB operations + pure	KB QUERY, findall, list sort by key	Evidence set	Ranked hypotheses	Proportional to evidence coverage
5. Assess coverage	Pure primitives	bitset count, vdr div	Evidence dimensions checked	Coverage fraction	Coverage $\times$ source confidence
6. Report	KB operations	KB ASSERT	Top hypotheses with scores	Ranked conclusions	Composite of evidence and coverage

C.3 Abductive Mode — Detailed Tool Chain

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
1. Assert observations	KB operations	KB ASSERT	Symptoms, anomalies	Observation facts	Direct observation: high
2. Write causal rules	KB operations	KB ASSERT	Domain knowledge	Cause→effect rules	Domain knowledge confidence

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
3. Query explanations	KB operations	KB QUERY, findall	Observations as goals	Possible causes	Per-rule confidence
4. Gather discriminating evidence	External + pure	net fetch, ENV EXEC	Targeted probes	New observations	Source-dependent
5. Requery with new evidence	KB operations	KB QUERY	Expanded fact set	Narrowed causes	Increases with evidence
6. Compute correlations	Python + pure	ENV EXEC, stat mean, vdr div	Time series data	Correlation scores	Exact fraction
7. Rank by evidence fit	Pure + KB	list sort by key, KB ASSERT	Causes + correlation scores	Best explanation	Product of evidence scores

C.4 Analogical Mode — Detailed Tool Chain

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
1. Assert source domain	KB operations	KB ASSERT	Known domain structure	Source domain facts and relations	fraction(1,1) if established knowledge

Step	Tool Category	Specific Tools	Input	Output	Confidence Effect
2. Assert target domain	KB operations	KB ASSERT	Unfamiliar domain observations	Target domain partial facts	Observation confidence
3. Write mapping rules	KB operations	KB ASSERT	Structural correspondence criteria	Mapping rules	fraction(1,1) for rule structure
4. Query mappings	KB operations	KB QUERY, findall	Source-target pairs	Structural correspondences	Per-mapping match quality
5. Score analogy strength	Pure primitives	vdr div, list length	Matched vs total relations	Analogy coverage score	Exact fraction: matched/total
6. Transfer conclusions	KB operations	KB ASSERT	Source domain conclusions + mappings	Hypothesized target conclusions	Analogy strength $\times$ source confidence
7. Verify transfers	Mixed	KB QUERY, ENV EXEC	Transferred conclusions	Verification results	Upgraded if verified, degraded if contradicted

C.5 Mode Composition Patterns

Composition	Flow	Use Case	Example
Abductive → Deductive	Generate hypotheses, then derive implications of each	Diagnosis then prediction	"DB connections caused the outage" → "then restarting the pool should fix latency"

Composition	Flow	Use Case	Example
Abductive → Inductive	Generate hypotheses, then score against evidence	Diagnosis with ranking	Generate 3 possible causes, gather evidence, rank
Inductive → Deductive	Observe pattern, then derive conse-quences	Discovery then application	"These 5 papers all use technique X" → "X should also work for problem Y because…"
Deductive → Inductive	Derive predictions, then check empirically	Theory testing	"If the rule is correct, then metric Z should decrease" → measure Z
Analogical → Deductive	Map structure from known domain, then deduce in new domain	Knowledge transfer	"Queues behave like pipes in fluid dynamics" → apply flow equations
Abductive → Inductive → Deductive	Full investi-gation cycle	SRE incident	Hypothesize causes → score against metrics → derive remediation steps

C.6 Mode Selection Heuristics

Problem Characteristics	Recommended Primary Mode	Indicator
Known rules, need conclusion	Deductive	User provides or system has formal rules
Observations without explanation	Abductive	"Why is X happening?"
Data without pattern	Inductive	"What's the trend in X?"
Familiar domain, unfamiliar instance	Analogical	"This looks like the time when…"

Problem Characteristics	Recommended Primary Mode	Indicator
Clear premises, verify implication	Deductive	"If A and B, does C follow?"
Multiple possible causes	Abductive → Inductive	"What caused X? Which is most likely?"
Sparse data, rich prior knowledge	Analogical → Deductive	"Based on similar systems…"
No prior knowledge, abundant data	Inductive → Abductive	"What patterns exist? What explains them?"

C.7 External Source Confidence Assignments

Source Type	Default Confidence	Adjustable?	Rationale
Prometheus/monitoring metric (live)	fraction(95, 100)	Yes	Instrumentation can have gaps or lag
Prometheus/monitoring metric (historical)	fraction(90, 100)	Yes	Retention and aggregation may lose detail
Database query result	fraction(98, 100)	Yes	Direct read from source of truth
REST API response	fraction(85, 100)	Yes	Depends on API reliability and staleness
Web search result	fraction(50, 100)	Yes	Unverified, potentially outdated
Peer-reviewed paper (claim)	fraction(80, 100)	Yes	Peer review provides some verification
User-stated fact	fraction(70, 100)	Yes	Not independently verified
LLM-generated content	fraction(30, 100)	Yes	Token prediction, not computation
Exact VDR computation	fraction(1, 1)	No	Mathematically guaranteed

Source Type	Default Confidence	Adjustable?	Rationale
Prolog derivation (from exact premises)	fraction(1, 1)	No	Logically guaranteed
Python script output (deterministic)	fraction(95, 100)	Yes	Script could have bugs

---

Appendix D: External Data Integration Pipeline Reference

D.1 Pipeline Stages by Source Type

Source Type	Acquire	Parse	Convert	Store	Index	Process
Prometheus JSON	net fetch	parse json, dict get	to fraction per value	KB ASSERT time-series binding	ring write for rolling window	stat mean, list filter, Python correlation
REST API JSON	net fetch	parse json	to fraction for numerics, atom for strings	KB ASSERT per record	lru push for recent records	list sort, list filter, dict operations
CSV file	fs read or ENV DOWNLOAD	parse csv	to fraction per cell, to number for ints	KB ASSERT per row or batch	bitset for rows processed	stat mean, stat percentile, list group by
Log file (text)	fs read or ENV DOWNLOAD	string split by newline	No conversion (stays as atoms)	KB ASSERT per parsed line	lru push for recent entries, ring write	string contains, list filter, list count
Database query	ENV EXEC (psql or script)	parse json or parse csv	to fraction for numerics	KB ASSERT per row	lru push, counter inc for counts	list sort, set operations, Prolog rules

Source Type	Acquire	Parse	Convert	Store	Index	Process
Web search	web search (external)	string operations on snippets	Atom storage for text	KB ASSERT with source provenance	lru push for sources	Prolog fact comparison, dedup via set intersection
Git/version control	ENV EXEC (git commands)	string split, string contains	No conversion	KB ASSERT for commits, diffs	ring write for recent commits	list filter by date, string contains for keywords
Process stdout	ENV EXEC (async)	string split by newline, parse per line	to fraction where applicable	STORE RESULT to KB path	ring write for chunked output	string contains for error detection, list filter

D.2 Conversion Boundary Records

When external float data enters the exact system, a conversion boundary fact is asserted:

```
ConversionBoundary = struct {

source_path: Text,        // where the raw data came from

source_type: Text,        // "prometheus_float", "api_double", "csv_decimal"

original_representation: Text, // "0.04237" as string

converted_value: fraction,    // fraction(4237, 100000)

conversion_method: Text,     // "decimal_string_to_fraction"

max_error: fraction,       // fraction(0, 1) for exact decimal, bounded for re

turn: i32,

notebook: Text,             // which inference notebook

};
```

D.3 Freshness Tracking

Freshness Level	Age	Constraint	Appropriate For
Real-time	< 1 minute	must refresh before use if stale	Live incident triage
Recent	< 15 minutes	acceptable for ongoing investigation	Active monitoring correlation
Session-fresh	Acquired this session	acceptable for most analysis	Research, planning
Historical	Any age, timestamp known	must declare age in provenance	Trend analysis, baselines
Stale	Age unknown or > 24 hours	warn before use, never use as sole evidence	Background context only

```
constraint("evidence_freshness", operational, active,
condition(forall(evidence(E),
    (evidence_acquired_at(E, T),
     current_time(Now),
     Now - T < max_staleness(E)))),
on_violation("warn"))
```

Appendix E: Confidence Propagation Rules

E.1 Propagation by Inference Step Type

Step Type	Input Confidences	Output Confidence	Formula	Rationale
Exact VDR computation	Any inputs	fraction(1, 1)	Fixed	Math is exact
Prolog derivation (all premises exact)	$C_1, C_2, \ldots, C_n$	$\min(C_1, \ldots, C_n)$	Weakest link	Conclusion is only as strong as weakest premise
Prolog derivation (mixed premises)	$C_1, C_2, \ldots, C_n$	$\min(C_1, \ldots, C_n)$	Weakest link	Same — derivation doesn't strengthen inputs
Evidence gathering (single source)	Source confidence Cs	Cs	Inherited	Data carries its source's reliability
Evidence gathering (multiple sources agree)	$C_1, C_2, \ldots, C_n$	$1 - \prod(1 - C_i)$	Independent confirmation	Multiple sources agreeing increases confidence
Evidence gathering (sources conflict)	$C_1, C_2, \ldots, C_n$	$\max(C_i)$ - conflict penalty	Penalized best	Conflict degrades even the strongest source
Inductive scoring	Coverage $\times$ mean evidence confidence	coverage $\times$ mean(C_i)	Proportional	Partial evidence gives partial confidence
Abductive ranking	Best explanation evidence / total evidence	evidence ratio $\times$ $\min(C_i)$	Coverage weighted	Explanation covers fraction of observations
Analogical transfer	Analogy strength $\times$ source confidence	strength $\times$ Csource	Product	Weaker analogy degrades transferred knowledge
Python computation	Input data confidence	min(inputs) $\times$ fraction(95, 100)	Degraded weakest link	Script could have bugs

Step Type	Input Confidences	Output Confidence	Formula	Rationale
LLM assessment step	—	fraction(30, 100)	Fixed floor	LLM judgment is unreliable
User-provided correction	—	fraction(70, 100)	Fixed	User might be wrong but probably right

E.2 Confidence Computation Examples

Scenario	Inputs	Computation	Result
Prometheus metric through Prolog rule	Prometheus: 95/100, Rule: 1/1	min(95/100, 1/1)	95/100
Two Prometheus sources agree	Source A: 95/100, Source B: 95/100	1 - (1-95/100)(1-95/100) = 1 - 1/400	399/400
Three sources, one conflicts	A: 95/100, B: 90/100, C: 95/100 (conflicts B)	max(90,95)/100 - 10/100 = 85/100	85/100
Inductive with 7/10 dimensions checked	Coverage: 7/10, mean evidence: 90/100	7/10 × 90/100	63/100
Analogy with 80% structural match	Strength: 80/100, source domain: 95/100	80/100 × 95/100	76/100
Full chain: Prometheus → Python → Prolog	95/100 → min(95/100) × 95/100 → min result	95/100 × 95/100 = 9025/10000	~90/100

E.3 Confidence Thresholds for Action

Confidence Range	Label	Appropriate Action
95/100 — 1/1	High	Act on conclusion directly
80/100 — 94/100	Moderate	Act with monitoring, note uncertainty
60/100 — 79/100	Low	Gather more evidence before acting

Confidence Range	Label	Appropriate Action
40/100 — 59/100	Speculative	Present as hypothesis, not conclusion
< 40/100	Unreliable	Do not present as conclusion, flag for investigation

These thresholds are configurable via constraints on the inference notebook.

Appendix F: Backtracking Reference

F.1 Backtrack Triggers

Trigger	Detection Method	State Before Backtrack
Execution failure	Command token returns error	Current step failed, partial state
Contradictory evidence	New fact contradicts existing fact in same notebook	Evidence conflict detected by Prolog
Hypothesis eliminated	All evidence contradicts the leading hypothesis	Prolog query returns no matching causes
Stall detection	steps since new evidence > stall threshold	No progress, resources being consumed
Confidence collapse	Conclusion confidence drops below threshold after new evidence	New evidence weakened the chain
Budget approaching	Budget counter > 80% of max with no conclusion in sight	Running out of resources
User redirect	User explicitly says “try a different approach”	User override

F.2 Backtrack Procedure

Step	Action	Tool	KB Effect
1	Record why current path failed	KB ASSERT	Failure reason stored for future avoidance
2	Push current path to LRU of attempted approaches	lru push	Prevents revisiting same approach
3	Pop investigation stack to previous branch point	stack pop	Restores earlier state reference

Step	Action	Tool	KB Effect
4	Check for alternative approaches at branch point	KB QUERY on unexplored alternatives	May find untried hypotheses or methods
5a	Alternative found: push new approach, transition to formalize	stack push, transition	New direction established
5b	No alternative: pop again (recursive backtrack)	stack pop	Walk further back
5c	Stack empty: transition to halt	—	No more options
6	Log backtrack event	KB ASSERT	Backtrack provenance recorded

F.3 Backtrack State Preservation

State Component	Preserved on Backtrack	Discarded on Backtrack	Rationale
Evidence facts (confirmed)	Yes	—	Confirmed evidence is still valid
Evidence facts (speculative)	—	Yes	Speculation specific to abandoned path
Prolog rules (general domain)	Yes	—	Domain knowledge transcends paths
Prolog rules (path-specific)	—	Yes (retracted)	Rules specific to abandoned hypothesis
Data primitive states	Partially — counters preserved, path-specific LRU entries expire	Path-specific entries	General progress preserved, path-specific context cleared

State Component	Preserved on Backtrack	Discarded on Backtrack	Rationale
External data (already fetched)	Yes	—	Don't re-fetch, reuse from KB
Step queue	Cleared	Yes	Old plan is invalid
Findings LRU	Preserved	—	Past findings may inform new path

Appendix G: Notebook Template Reference

G.1 Template: SRE Incident Investigation

Field	Pre-populated Value	User Provides
inference type	abductive	—
goal	root cause(,) fact asserted with confidence > 80/100	—
initial step queue	[1. pull alerting metric, 2. pull correlated metrics, 3. check recent deployments, 4. check dependency health, 5. formalize causal hypotheses, 6. test leading hypothesis, 7. confirm root cause, 8. write remediation plan, 9. write prevention rules]	Symptom description
data primitives	counter(queries issued, max=20), counter(hypotheses tested), lru(findings, 30), lru(sources, 20), bitset(dependencies checked, N), lock(investigating), queue(remediation steps, 10), ring(metric snapshots, 60)	—
Prolog preloaded	General SRE causal rules from root.ops.sre rules	Incident-specific symptoms
external sources	Prometheus, deployment API, dependency health endpoints	Source URLs
constraints	query budget < 20, max steps < 50, evidence freshness < 15min	Override if needed

G.2 Template: Programming Bug Investigation

Field	Pre-populated Value	User Provides
inference type	abductive	—
goal	root cause(,) and fix verified(true)	Failing test or error description
initial step queue	[1. reproduce failure, 2. check recent failures for pattern, 3. map dependency graph, 4. trace execution path, 5. identify divergence, 6. formalize hypothesis, 7. write fix, 8. verify fix, 9. write regression test]	—
data primitives	lru(recent failures, 20), stack(undo stack, 10), stack(fix steps, 10), bitset(resolved bugs, 50), lock(editing module), counter(retry count, max=3)	—
Prolog preloaded	Code dependency rules if available, code smell rules	Code-specific facts
external sources	Source files, test runner, debugger output	File paths, test commands
constraints	retry count < 3 per approach, editing lock required before code changes	—

G.3 Template: Research Compilation

Field	Pre-populated Value	User Provides
inference type	inductive	—
goal	ranked approaches(,) with evidence coverage > 70/100	Research question
initial step queue	[1. broad search, 2. categorize findings, 3. deep dive per category, 4. score by evidence quality, 5. identify gaps, 6. fill gaps with targeted search, 7. rank and synthesize]	—
data primitives	lru(papers, 50), counter(papers found), counter(papers analyzed), queue(to analyze, 30), bitset(themes covered, 10), ring(methodology notes, 20), lock(analysis active)	—
external sources	Web search, academic APIs	Topic keywords

Field	Pre-populated Value	User Provides
constraints	minimum 3 peer-reviewed sources per major claim, evidence coverage > 7/10	—

G.4 Template: Decision Matrix

Field	Pre-populated Value	User Provides
inference type	deductive	—
goal	ranked options(,) with sensitivity analysis complete	Options and criteria
initial step queue	[1. collect criteria and weights, 2. verify weights sum to 1, 3. score each option, 4. compute weighted totals, 5. sensitivity analysis, 6. present with robustness assessment]	—
data primitives	counter(options scored), counter(criteria count), ring(discussion points, 20), lru(data sources, 10)	—
constraints	weights sum to one (axiom), all options scored on all criteria	—

G.5 Template: Argument Construction

Field	Pre-populated Value	User Provides
inference type	deductive + inductive	—
goal	argument structure valid and unsupported claims = 0	Thesis/position
initial step queue	[1. state thesis, 2. search for supporting evidence, 3. formalize argument structure, 4. check for unsupported claims, 5. search for counter-arguments, 6. prepare rebuttals, 7. order for persuasive flow, 8. generate text following structure]	—

Field	Pre-populated Value	User Provides
data primitives	queue(argument order, 20), lru(sources, 30), counter(unsupported claims), counter(counter arguments addressed), bitset(evidence dimensions, 10)	—
constraints	every claim has source with strength > peer reviewed, every known objection has a rebuttal	—

Appendix H: Contradiction Detection Reference

H.1 Contradiction Types

Type	Detection Method	Example	Resolution Strategy
Direct factual	Two facts assert different values for same predicate and args	bob age(32) and bob age(59) in same scope	Check provenance — which source is more reliable
Logical	Prolog derives both P and not(P)	reachable(a,d) and not(reachable(a,d))	Rules are inconsistent — inspect rule set
Statistical	Two evidence sources give significantly different values	Source A: error rate 1%, Source B: error rate 15%	Investigate measurement methodology difference
Temporal	Fact was true at time T1 but false at time T2	server status(healthy) at 14:00, server status(down) at 14:30	Not a contradiction — state changed. Record transition.
Cross-notebook	Two notebooks reach opposite conclusions from overlapping evidence	Notebook A: cause is DB. Notebook B: cause is network.	Compare evidence sets and reasoning chains

H.2 Contradiction Detection Rules

```
Rule: direct_contradiction(Fact1, Fact2) :-
fact(Predicate, Args1, Value1, Notebook1),
fact(Predicate, Args2, Value2, Notebook2),
Args1 = Args2,
Value1 \= Value2,
same_scope(Notebook1, Notebook2).
Rule: logical_contradiction(Fact, NegFact) :-
derived(Fact, Chain1),
derived(NegFact, Chain2),
negation(Fact, NegFact).
Rule: statistical_contradiction(Metric, Source1, Value1, Source2, Value2) :-

evidence(Metric, Source1, Value1, _),
evidence(Metric, Source2, Value2, _),
Source1 \= Source2,
abs(Value1 - Value2) > Value1 * fraction(5, 10).
Rule: cross_notebook_contradiction(Conclusion1, Notebook1, Conclusion2, Note

conclusion(Conclusion1, Notebook1, _, _),
conclusion(Conclusion2, Notebook2, _, _),
Notebook1 \= Notebook2,
contradicts(Conclusion1, Conclusion2).
```

H.3 Contradiction Resolution Strategies

Strategy	When To Use	Procedure	Outcome
Source reliability	Different sources disagree	Compare provenance weights, trust higher	Weaker source's fact downgraded
Temporal resolution	Values changed over time	Assert both with timestamps, mark transition	Not a contradiction — state change
Scope separation	Facts true in different contexts	Verify facts belong to different scope KBs	Not a contradiction — different contexts
Evidence gathering	Insufficient data to resolve	Acquire additional evidence to discriminate	One side gains support
Rule revision	Logical rules produce contradiction	Inspect rule set for over-broad rules	Rule modified or retracted
Confidence degradation	Cannot resolve	Degrade confidence of both conclusions	Both marked as uncertain
Escalation	System cannot resolve	Flag for human review	User decides

---

Appendix I: Inference Provenance Schema

I.1 Conclusion Record Structure

```
InferenceConclusion = struct {

id: Text,                    // unique conclusion identifier

notebook: Text,                // dotted path of inference notebook

statement: Fact,               // the conclusion itself (Prolog fact)

inference_mode: enum,            // deductive, inductive, abductive, analogical

confidence: fraction,              // exact VDR fraction

// Derivation chain

derived_from: []EvidenceRef,      // evidence facts this conclusion depends on

via_rules: []RuleRef,            // Prolog rules used in derivation
```

```
via_tools: []CommandRef,        // command tokens that produced evidence
via_scripts: []ScriptRef,       // Python scripts involved
// Alternatives
alternatives_considered: []AlternativeRef,  // other hypotheses tested
// Metadata
concluded_at: i32,                  // turn number
steps_to_reach: i32,           // total loop iterations
external_queries_used: i32,     // Prometheus/API queries
backtrack_count: i32,         // times the investigation backtracked
};
```

I.2 Evidence Reference Structure

```
EvidenceRef = struct {
fact_id: Text,                  // reference to KB fact
source_type: Text,            // "prometheus", "user", "prolog_derivation", e
source_detail: Text,           // URL, user ID, rule name
confidence: fraction,          // source confidence at time of acquisition
acquired_at: i32,                 // turn when acquired
conversion_boundary: ?ConversionBoundary,  // if external data was converted
};
```

I.3 Provenance Query Patterns

Query	Purpose	Returns
conclusion derivation(ConclusionId, Chain)	Full derivation chain for a conclusion	Ordered list of evidence → rules → intermediate → conclusion
conclusion evidence(ConclusionId, Evidence)	All evidence supporting a conclusion	List of evidence refs with confidences
conclusion weakest link(ConclusionId, Weakest)	Least confident input	Evidence ref with lowest confidence
conclusion external sources(ConclusionId, Sources)	External sources used	List of Prometheus queries, API calls, etc.
conclusion alternatives(ConclusionId, Alts)	What else was considered	Alternative hypotheses with their scores
conclusion would change if(ConclusionId, FactId, Impact)	Impact of removing one piece of evidence	Whether conclusion still holds, new confidence
notebook conclusions(NotebookPath, Conclusions)	All conclusions from a notebook	List of conclusion records
cross notebook support(Conclusion, SupportingNotebooks)	Which other notebooks support this	Notebooks with compatible conclusions
challenge(ConclusionId, CounterFact)	What happens if counter-fact is asserted	Re-evaluated conclusion or contradiction

---

Appendix J: Resource Budget Reference

J.1 Budget Parameters

Budget	Counter Name	Default Max	Configurable?	On Exhaustion
Total steps	steps executed	50	Yes, per notebook	Conclude with partial findings or halt
External queries	queries issued	20	Yes, per notebook	No more external data, work with what exists

Budget	Counter Name	Default Max	Configurable?	On Exhaustion
Python executions	scripts executed	10	Yes, per notebook	No more scripts, use pure primitives only
Backtrack depth	investigation path.size	15	Yes, per notebook	No more branching, conclude current path
Sub-notebooks	child count	5	Yes, per notebook	No more branching, inline investigation
Hypothesis count	hypotheses tested	20	Yes, per notebook	Stop generating hypotheses, rank existing
Stall tolerance	steps since new evidence	5	Yes, per notebook	Force backtrack or conclude
Wall clock	elapsed minutes	30	Yes, per notebook	Hard stop, conclude or halt

J.2 Budget Allocation for Sub-Notebooks

When a notebook spawns a child notebook, the child receives a fraction of the parent's remaining budget:

Parent Remaining	Child Allocation	Rationale
> 80% of total	40% of remaining	Early in investigation, generous allocation
50-80% of total	25% of remaining	Mid-investigation, moderate allocation
< 50% of total	15% of remaining	Late, conservative allocation
< 20% of total	No child allowed	Not enough budget to branch

All allocations are exact VDR fractions of the remaining integer budget. The parent's budget is decremented by the child's allocation.

Appendix K: Cross-Domain Inference Pattern Comparison

K.1 Tool Usage Frequency by Domain

Tool Category	SRE	Bug Fix	Architecture	Speech	Email	Research	Decision	Worldbuild	Teaching	Data Quality	Planning
KB ASSERT (facts)	High	High	Medium	High	High	High	Medium	High	Medium	High	Medium
KB ASSERT (rules)	High	Medium	High	Medium	High	Medium	High	High	Medium	High	High
KB QUERY	High	High	High	Medium	High	High	High	High	High	High	High
net fetch	High	Low	Medium	Medium	Low	High	Low	Low	Low	Medium	Low
ENV EXEC (Python)	High	High	Medium	Low	Low	Medium	Medium	Low	Medium	High	Low
parse json	High	Low	Low	Low	Low	Medium	Low	Low	Low	Medium	Low
string split/filter	Medium	High	Low	Low	High	Medium	Low	Low	Low	Medium	Low
list sort/filter	Medium	Medium	Medium	Medium	Medium	High	High	Low	Low	High	Medium
stat mean/percentile	High	Low	Medium	Low	Low	Medium	Medium	Low	Low	High	Low
graph *	Low	High	Medium	Low	Low	High	Low	Low	Low	Low	High
vdr * arithmetic	High	Low	High	Low	Low	High	High	Low	Medium	High	Medium

K.2 Data Primitive Usage by Domain

Primitive	SRE	Bug Fix	Architecture	Speech	Email	Research	Decision	Worldbuild	Teaching	Data Quality	Planning
lru	findings, sources	recent failures, fixes	—	sources	key statements	papers, sources	data sources	recent events	student answers	findings	—

Primitive	SRE Bug Fix	Architecture	Speech	Email	Research	Decision	Worldbuilding	Teaching	Data Quality	Planning
counter	queries, hypotheses	failures, retries	scored claims	unsupported proposals	sources found, analyzed	criteria	violations, chapters	steps	tasks	
lock	investigating	—	—	—	analysis active	—	timeline	—	active	deployment gate
queue	remediation options		argument order	—	to analyze	—	—	—	check queue	task queue
stack	investigation path	—	—	—	—	—	plot threads	hints	investigation path	blocked
ring	metric snapshots	—	—	—	methodology notes	discussion	—	—	—	daily progress
bitset	deps resolved checked	—	evidence dims	processed	themes	—	characters	concepts	sources cleared	milestones

K.3 Average Inference Complexity by Domain

Domain	Typical Steps	Typical External Queries	Typical Backtracks	Typical Sub-Notebooks	Typical Confidence
SRE incident	15-30	5-15	1-3	0-2	85-95/100
Bug investigation	10-20	2-5 (test runs)	1-2	0-1	90-100/100
Architecture design	8-15	2-5	0-1	0	75-90/100
Speech construction	10-20	3-8	0-1	0	70-85/100
Email analysis	5-15	0	0	0	80-95/100

Domain	Typical Steps	Typical External Queries	Typical Backtracks	Typical Sub-Notebooks	Typical Confidence
Research compilation	15-30	5-15	1-2	1-3	60-80/100
Decision matrix	8-12	1-3	0	0	85-95/100
Worldbuilding	5-10 per chapter	0	0	0-1	95-100/100 (rules-based)
Teaching	5-15 per concept	0-2	0-1	0	90-100/100
Data quality	10-25	3-10	1-2	0-1	80-95/100
Project planning	8-15	1-3	0-1	0	85-95/100

Appendix L: Inference Loop Interaction with Session Management

L.1 Inference Across Session Boundaries

Scenario	Mechanism	State Preserved	State Lost
Notebook active, session saved	session snapshot captures all notebook data primitives	Counters, queues, stacks, LRUs, bitsets, locks, investigation state	Nothing — full capture
Notebook active, session reset	All live state cleared	KB facts, rules, constraints (persistent)	All data primitive states — investigation progress lost
Notebook concluded, session saved	Conclusion is persistent KB fact, notebook archived	Conclusion, evidence, provenance	Live data primitives (no longer needed)
Notebook active, clone killed	Persistent KB assertions survive	Committed evidence and conclusions	In-progress investigation state

L.2 Inference in Disposable Clones

Orchestrated inference in a disposable clone follows a specific pattern:

Phase	Action	Persistent?	Survives Kill?
1. Receive task	Read problem from persistent KB	—	—
2. Create notebook	KB structure is persistent, data primitives are live	Mixed	KB yes, primitives no
3. Investigate	Assert evidence, write rules, execute tools	Evidence facts: persistent	Yes
4. Conclude	Assert conclusion with provenance	Conclusion: persistent	Yes
5. Clone killed	Live state destroyed	—	Conclusion survives
6. New clone	Reads conclusion from persistent KB	—	—

The key insight: the notebook's persistent facts (evidence, rules, conclusions) survive clone recycling. The live state (counters, queues, investigation path) does not. A new clone can pick up where the old one left off by reading the persistent facts — but it loses the investigation's working memory.

For long investigations that might span multiple clone lifetimes, the LLM should periodically promote critical live state to persistent facts:

```
CMD: KB_ASSERT(root.inference.incident_002.checkpoint,

fact(investigation_progress(

    steps_completed(23),

    hypotheses_tested(4),

    leading_hypothesis("db_connection_exhaustion"),
```

```
    remaining_steps(["verify with Prometheus", "write remediation"])))
```

L.3 Concurrent Inference Notebooks

Multiple notebooks can be active simultaneously in different scope branches. Isolation and interaction rules:

Aspect	Same-Branch Notebooks	Cross-Branch Notebooks
Fact visibility	Child sees parent's facts (inheritance)	Not visible unless mounted or cross-queried
Data primitive visibility	Child can read parent's primitives (inheritance)	Not visible
Lock coordination	Parent lock visible to child — useful for mutual exclusion	Not visible — independent
Evidence sharing	Natural through inheritance	Requires explicit KB ASSERT to shared KB or mount
Contradiction detection	Automatic within scope	Requires cross notebook contradiction query
Budget	Child draws from parent's budget	Independent budgets

Appendix M: LLM Failure Mode Reference

M.1 Orchestration Errors and Mitigations

Failure Mode	Description	Detection	Mitigation
Wrong mode selection	LLM chooses deductive when abductive is appropriate	Deduction returns no results despite evidence existing	Stall detection triggers backtrack, LLM reassesses
Bad Prolog rules	LLM writes rules that don't capture the domain correctly	Query returns surprising results, or no results when expected	Contradiction detection, evidence checking against conclusions
Missed evidence source	LLM doesn't query a relevant external source	Coverage bitset shows unchecked dimensions	Coverage constraint warns when bitset count / width < threshold

Failure Mode	Description	Detection	Mitigation
Over-confident conclusion	LLM concludes too early with insufficient evidence	Low evidence count relative to problem complexity	Minimum evidence constraint on conclude transition
Infinite loop	LLM keeps gathering evidence without converging	Stall counter exceeds threshold	Stall constraint forces backtrack or conclude
Wrong hypothesis ranking	Inductive scoring uses bad criteria	Leading hypothesis contradicted by subsequent evidence	Backtrack trigger on contradictory evidence
Premature backtrack	LLM abandons a correct path due to one confusing data point	Correct path appears in attempted steps LRU	Budget management — don't exhaust budget on first path
Script bugs	LLM writes Python with errors	Script returns error or nonsensical results	try catch in execution, error logged, alternative approach
Path reference error	LLM uses wrong dotted path	Command token validation catches non-existent path	Error logged in lru(recent command errors), LLM adjusts
Evidence misinterpretation	LLM asserts wrong Prolog fact from correct raw data	Downstream Prolog derivation contradicts other evidence	Contradiction detection, provenance trace to misinterpretation

M.2 Error Detection by Loop Phase

Loop Phase	Detectable Errors	Detection Mechanism
Assess	Wrong next step selection, missed obvious gap	Stall detection after multiple assess cycles with no new evidence
Formalize	Bad Prolog rules, buggy Python, wrong primitive	Execution failure in next phase, or surprising results
Execute	Tool failure, timeout, resource error	Command token error handling, exit codes

Loop Phase	Detectable Errors	Detection Mechanism
Store	Wrong location, type mismatch, overwrite of good data	Path validation, type checking, mutation log
Branch	Unnecessary branching, too-deep nesting	Depth constraint, budget allocation limits
Backtrack	Premature abandon, failure to backtrack when needed	Stall detection (should have backtracked), contradiction (should have backtracked)
Conclude	Premature conclusion, over-confident, unsupported claims	Minimum evidence constraint, confidence threshold, coverage check

M.3 Error Recovery Patterns

Error Pattern	Recovery Sequence	Data Primitive Support
Wrong path, need backtrack	Pop stack, check LRU for untried approaches, push new path	stack (investigation), lru (attempted)
Bad evidence, need correction	Retract wrong fact, note in LRU, re-gather	lru (findings update), counter (retraction count)
Script failure, need alternative	Log error, try pure primitives instead, or rewrite script	lru (errors), counter (retries)
Budget nearly exhausted	Assess remaining budget, prioritize highest-impact remaining step	counter (budget tracking), queue (prioritized remaining)
Repeated failure on same step	After 3 retries, skip step, note as unresolvable, continue	counter (retry per step), bitset (skipped steps)

Appendix N: Cumulative System Statistics

N.1 Complete Paper Series

Paper	Registry	Central Result
VDR-1	[HOWL-VDR-1-2026]	Exact arithmetic in irreducible triple form
VDR-2	[HOWL-VDR-2-2026]	15 domains, 282 tests
VDR-3	[HOWL-VDR-3-2026]	23 domains, transcendental integration

Paper	Registry	Central Result
VDR-4	[HOWL-VDR-4-2026]	24-module ML stack, working exact transformer
VDR-5	[HOWL-VDR-5-2026]	Prolog KB architecture, constraints, scoped knowledge
VDR-6	[HOWL-VDR-6-2026]	255 primitives, command tokens, operational environments
VDR-7	[HOWL-VDR-7-2026]	12-phase lifecycle, training through retirement
VDR-8	[HOWL-VDR-8-2026]	Data primitives, dotted paths, session management
VDR-9	**[HOWL-VDR-9-2026]**	**Orchestrated Inference: structured reasoning through tool composition**

N.2 System Capability Progression

Paper	What It Added	Cumulative Capability
VDR-1–4	Exact arithmetic + ML stack	Can compute exactly
VDR-5	Knowledge bases, constraints, scoping	Can know and constrain
VDR-6	Primitives, commands, environments	Can do and execute
VDR-7	Lifecycle management	Can train, deploy, and retire
VDR-8	Runtime state, addressing, sessions	Can remember, address, and recover
VDR-9	Orchestrated Inference	Can investigate, reason, and conclude

N.3 Unchanged Counts from VDR-8

Component	Count	Source	Changed by VDR-9?
Pure primitives	289	VDR-6 + VDR-8	No
Operational primitives	44	VDR-6	No
Total primitives	333	VDR-6 + VDR-8	No
KB struct fields	24	VDR-8	No
Modules	37	VDR-8	No
Existing tests	705	VDR-1–4	No
Planned tests	1221	VDR-6–8	No

VDR-9 adds no new primitives, no new struct fields, and no new modules. It specifies patterns of use, not new capabilities. Test additions come from testing the inference

notebook schema, loop invariants, confidence propagation, and contradiction detection — estimated at 80-120 additional tests.

END VDR-9 EXTENDED APPENDIX TABLES

[**HOWL-VDR-9-2026**] Orchestrated Inference. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOW
VDR-9-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-VDR-2-2026**] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026

[**HOWL-MATH-3-2026**] The Transcendental Hierarchy. Github: https://github.com/ghowland/papers/tree/main/papers/M
MATH-3-2026

[**HOWL-MATH-4-2026**] The Universal Power-of-Two Basis. Github: https://github.com/ghowland/papers/tree/main/pape
MATH-4-2026

[**HOWL-VDR-3-2026**] VDR Gym Extension. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-3-2026

[**HOWL-VDR-4-2026**] Exact-Fraction Language Model Architecture. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026

[**HOWL-LLM-1-2026**] Integer LLM with Prolog Knowledge Base. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026

[**HOWL-VDR-5-2026**] Exact Arithmetic Meets Logical Provenance. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026

[**HOWL-VDR-6-2026**] Computational Primitives and Operational Environments. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026

[**HOWL-VDR-7-2026**] Complete Lifecycle Technical Specification. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026

[**HOWL-VDR-8-2026**] Computational State Primitives, Universal Data Pathing, and Session Management. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026